UNIVERSITAT POLITÈCNICA DE CATALUNYA
BARCELONATECH
Facultat d'Informàtica de Barcelona

FIB

SCALING AI-DRIVEN MARKET SHARE ESTIMATION TO PRODUCTION: ARCHITECTURE, PIPELINES AND PRODUCTIZATION OF MARKET SHARE

JOAN BENNÀSSAR MARTÍN

Thesis supervisor
MAGDALENA SZTANDARSKA (SHALION DATA SERVICES SL)

Tutor: RAMON SANGÜESA SOLE (Department of Computer Science)

Degree
Bachelor's Degree in Artificial Intelligence

Bachelor's thesis

Facultat d'Informàtica de Barcelona (FIB)

Universitat Politècnica de Catalunya (UPC) - BarcelonaTech

26/06/2026

Abstract

This thesis describes the scaling of an AI-driven market share estimation service from a functional prototype to a production system running weekly for real clients. The work was carried out during an internship at Shalion, a Barcelona-based e-commerce analytics company, in the context of the launch of the Retail Media Maestro product, whose *Marketshare* module lets clients view estimates across the four most important US retailers, covering more than 20,000 seeds.

The starting point was a modular pipeline that infers product- and brand-level market share from public digital shelf signals — Best Seller Rankings, units-sold indicators, reviews, and ratings — and from confidential Amazon Vendor Central actuals where available. At the seed volume required by the new product, the system took more than 12 hours each week and did not complete, due to a Snowflake queuing problem caused by more than 500,000 individual database reads per execution. The primary engineering contribution of this thesis is the elimination of this bottleneck through a two-stage architectural redesign that reduces total reads to a single bulk query per client batch (around 24 per weekly run). As a result, the ECS computation for a single batch (approximately 2,500 seeds) now runs in 15–20 minutes and, with batches executing in parallel, the full weekly run completes in approximately one hour, where previously it did not finish at all.

In parallel, the modelling layer was extended. The original four-scenario orchestration was replaced by a nine-tier routing system that assigns each seed to the most appropriate pipeline based on the available signals. Two new XGBoost pipelines were designed, trained, and deployed for non-Amazon retailers and rank-only seeds, reducing held-out WMAPE from 101% to 46% and from 115% to 63% relative to the baseline models. A revenue proxy based on weekly review deltas was also developed and empirically validated to aggregate estimates from subcategory up to higher category levels, and a migration of the output storage to Apache Iceberg tables was designed and implemented on a development branch.

This thesis demonstrates that scaling an AI-based estimation service to production is itself a substantial technical contribution, requiring the integration of database engineering, cloud orchestration, machine learning, and cross-team coordination in a way that no single discipline alone could have achieved.

Resum

Aquesta tesi descriu l'escalat d'un servei d'estimació de quota de mercat basat en intel·ligència artificial, des d'un prototip funcional fins a un sistema en producció que s'executa setmanalment per a clients reals. El treball es va dur a terme durant unes pràctiques a Shalion, una empresa d'analítica de comerç electrònic amb seu a Barcelona, en el context del llançament del producte Retail Media Maestro, el mòdul de *Marketshare* del qual permet als clients consultar estimacions dels quatre retailers més importants dels Estats Units en més de 20.000 seeds.

El punt de partida era una pipeline modular que infereix la quota de mercat a nivell de producte i marca a partir de senyals públiques de les prestatgeries digitals — rànquings de bestsellers, indicadors d'unitats venudes, ressenyes i valoracions — i de dades reals confidencials d'Amazon Vendor Central quan estan disponibles. Al volum de seeds requerit pel nou producte, el sistema tardava més de 12 hores cada setmana i no completava l'execució, a causa d'un problema d'encuament a Snowflake generat per més de 500.000 lectures individuals de base de dades per execució. La contribució d'enginyeria principal d'aquesta tesi és l'eliminació d'aquest coll d'ampolla mitjançant un redisseny arquitectònic en dues etapes que redueix el total de lectures a una única consulta massiva per lot de client (al voltant de 24 per execució setmanal). Com a resultat, el càlcul a ECS d'un únic lot (aproximadament 2.500 seeds) s'executa ara en 15–20 minuts i, amb els lots en paral·lel, l'execució setmanal completa finalitza en aproximadament una hora, quan abans no acabava.

En paral·lel, es va ampliar la capa de modelatge. L'orquestració original de quatre escenaris va ser substituïda per un sistema de nou nivells que assigna cada seed a la pipeline més adequada segons els senyals disponibles. Es van dissenyar, entrenar i desplegar dues noves pipelines basades en XGBoost per als minoristes no-Amazon i les seeds amb només rànquing, reduint el WMAPE en dades de validació del 101% al 46% i del 115% al 63% respecte als models de partida. També es va desenvolupar i validar empíricament un proxy d'ingressos basat en deltes setmanals de ressenyes per agregar les estimacions des del nivell de subcategoria fins a nivells de categoria superiors, i es va dissenyar i implementar en una branca de desenvolupament una migració de l'emmagatzematge de sortida a taules Apache Iceberg.

La tesi demostra que portar a producció a escala un servei d'estimació basat en IA és en si mateix una contribució tècnica substancial, que requereix la integració d'enginyeria de bases de dades, orquestració al núvol, aprenentatge automàtic i coordinació entre equips d'una manera que cap disciplina individual per si sola no hauria pogut assolir.

Resumen

Esta tesis describe el escalado de un servicio de estimación de cuota de mercado basado en inteligencia artificial, desde un prototipo funcional hasta un sistema en producción que se ejecuta semanalmente para clientes reales. El trabajo se realizó durante unas prácticas en Shalion, una empresa de analítica de comercio electrónico con sede en Barcelona, en el contexto del lanzamiento del producto Retail Media Maestro, cuyo módulo de *Marketshare* permite a los clientes consultar estimaciones de los cuatro retailers más importantes de Estados Unidos en más de 20.000 seeds.

El punto de partida era una pipeline modular que infiere la cuota de mercado a nivel de producto y marca a partir de señales públicas de los escaparates digitales — rankings de los más vendidos, indicadores de unidades vendidas, reseñas y valoraciones — y de datos reales confidenciales de Amazon Vendor Central cuando están disponibles. Al volumen de seeds requerido por el nuevo producto, el sistema tardaba más de 12 horas cada semana y no completaba la ejecución, debido a un problema de encolamiento en Snowflake generado por más de 500.000 lecturas individuales de base de datos por ejecución. La contribución de ingeniería principal de esta tesis es la eliminación de este cuello de botella mediante un rediseño arquitectónico en dos etapas que reduce el total de lecturas a una única consulta masiva por lote de cliente (alrededor de 24 por ejecución semanal). Como resultado, el cálculo en ECS de un único lote (aproximadamente 2.500 seeds) se ejecuta ahora en 15–20 minutos y, con los lotes en paralelo, la ejecución semanal completa finaliza en aproximadamente una hora, cuando antes no terminaba.

En paralelo, se amplió la capa de modelado. La orquestación original de cuatro escenarios fue sustituida por un sistema de nueve niveles que asigna cada seed a la pipeline más adecuada en función de las señales disponibles. Se diseñaron, entrenaron y desplegaron dos nuevas pipelines basadas en XGBoost para los minoristas no-Amazon y las seeds con solo ranking, reduciendo el WMAPE en datos de validación del 101% al 46% y del 115% al 63% respecto a los modelos de partida. También se desarrolló y validó empíricamente un proxy de ingresos basado en deltas semanales de reseñas para agregar las estimaciones desde el nivel de subcategoría hasta niveles de categoría superiores, y se diseñó e implementó en una rama de desarrollo una migración del almacenamiento de salida a tablas Apache Iceberg.

Esta tesis demuestra que llevar a producción a escala un servicio de estimación basado en IA es en sí mismo una contribución técnica sustancial, que requiere la integración de ingeniería de bases de datos, orquestación en la nube, aprendizaje automático y coordinación entre equipos de una forma que ninguna disciplina individual por sí sola hubiera podido lograr.

Acknowledgements

I would like to express my sincere gratitude to all the people who have supported me throughout the development of this thesis.

First and foremost, I would like to thank my family and friends for being by my side, accompanying me and helping me throughout all my years of study. Their unconditional support, patience, and encouragement have been a constant throughout this process and far beyond it.

I am especially grateful to the Shalion team for providing the opportunity to work on a real industrial problem and for their continuous support. In particular, I would like to thank Magdalena Sztandarska for her guidance, availability, and valuable feedback during the project. I would also like to thank the members of the Data Science and AI team — David Roche, Alex Tort-Martorell, and Aleix Ortigosa — for their support and collaboration throughout the internship, and Lluc Furriols, who planted the seed of this project and whose prior work provided the foundation on which this thesis builds. Special thanks are also due to Aleix Pujol and Maria Barnils from the Data Engineering team, with whom I collaborated closely to design and deploy the new architecture, and who generously shared their knowledge on multiple occasions when it was needed most.

Finally, I would like to thank my academic tutor at UPC, Ramon Sangüesa Solé, for his supervision, insightful comments, and constructive guidance throughout the development of this thesis.

Contents

List of Figures

List of Tables

Acronyms and Key Terms

1P (First-Party) Sales fulfilled directly by the retailer (e.g. Amazon Retail). In the context of this thesis, 1P refers to units sold by Amazon acting as the direct seller on its own marketplace.

3P (Third-Party) Sales fulfilled by an independent seller through the retailer's marketplace. A product may be sold by both 1P and 3P channels simultaneously.

ASIN Amazon Standard Identification Number. Amazon's unique 10-character product identifier.

ASM Amazon Shelf Maestro. Shalion's product specialised in Amazon ecosystem monitoring (Buy Box, Prime eligibility, advertising placements).

AWS Glue Catalog Managed metastore on AWS used as the Iceberg catalog, tracking each write to an Iceberg table as an atomic snapshot (Section 5.5).

BSR Best Seller Ranking. The position assigned to a product within a category when the Product Listing Page is sorted by sales performance. The primary demand signal used throughout this thesis.

Buy Box The "Add to Cart" seller on an Amazon Product Detail Page. Indicates which channel (1P or 3P) is winning the sale at a given moment. Scraped hourly for Pipeline 1 clients to estimate the 1P/3P split.

CI Continuous Integration. Automated build and test pipeline triggered on every pull request to `develop` or `main`.

CTE Common Table Expression. A named intermediate result within a SQL query, used to structure the single bulk query that retrieves the weekly estimation panel (Section 5.4.3).

DAG Directed Acyclic Graph. The abstraction used by Apache Airflow to define and schedule the dependencies between tasks in a workflow.

DBT Data Build Tool. A transformation framework for the data warehouse. Used in this project to build and maintain the input tables consumed by the estimation pipeline.

DSM	Digital Shelf Maestro. Shalion's core digital shelf analytics product, monitoring categories daily across retailers.
ECS	Elastic Container Service. AWS's container orchestration service, used to run the Fargate tasks that execute Pipeline Complete.
EDA	Exploratory Data Analysis.
ETL	Extract, Transform, Load. A data integration pattern.
Fargate	AWS Fargate. A serverless compute engine for containers, used as the execution environment for each Pipeline Complete invocation in the child DAG.
GDPR	General Data Protection Regulation. EU Regulation 2016/679 on personal data processing.
GroupKFold	A cross-validation strategy that ensures all records from the same group (here: execution) are kept in the same fold, preventing data leakage.
IAM	Identity and Access Management. AWS service for managing access permissions; used for the Iceberg write role in Section 5.5.
IQR	Interquartile Range. The spread between the first and third quartiles, used as an outlier-filtering criterion before fitting the Pipeline 1 calibration.
KPI	Key Performance Indicator.
MAPE	Mean Absolute Percentage Error. A prediction error metric. Less robust than WMAPE for skewed market share distributions.
MLOps	Machine Learning Operations. The set of practices for deploying, monitoring, and maintaining machine learning systems in production.
MSM	Market Share Maestro. Shalion's dedicated market share product.
NS	NumberSells. A coarse units-sold indicator displayed by some retailers (e.g. "30K+ bought in past month"), used in Pipelines 1 and 2.
Optuna	An open-source hyperparameter optimisation framework based on tree-structured Parzen estimation, used to tune the XGBoost pipelines.
Parquet	A columnar storage file format used for the pipeline output contracts written to S3 and exposed to Snowflake via External Tables.
PDP	Product Detail Page. The individual product page showing buybox information, product variants, descriptions, and the Add to Cart button. Scraped 24 times per day for Pipeline 1 clients.
Pipeline	A per-seed estimation workflow. Receives one in-memory data slice and executes seven sequential steps to produce a market share output contract.

Pipeline Complete
: The entry point for a single Fargate task. Issues the bulk query, partitions the result by seed and location, and dispatches parallel Pipeline workers.

PLP
: Product Listing Page. The ranked grid of products that a retailer displays for a given subcategory query. The primary source of public demand signals and the unit from which a seed's weekly data is derived.

PSI
: Population Stability Index. A metric used in model monitoring to detect distribution shift between two populations (e.g. training vs. production).

PUE
: Power Usage Effectiveness. The ratio of total data-centre energy to computing energy, used in the carbon-footprint estimation (Section 8.4).

RSS
: Resident Set Size. The actual memory occupied by a process in RAM, used as the primary memory metric in the execution instrumentation.

S3
: Amazon Simple Storage Service. AWS object storage, used to hold the Parquet output contracts produced by the estimation pipeline.

RMM
: Retail Media Maestro. Shalion's cross-retailer retail media intelligence product, tracking more than 200,000 keywords daily across Amazon, Walmart, Target, and Kroger in the US.

Seed
: A Product Listing Page of a specific subcategory in a specific store. A seed may produce different product compositions depending on the location (delivery address or regional store). Market share is computed for a seed in a given location in a given week, based on the scraping carried out for that combination. The fundamental unit of estimation throughout this thesis.

SKU
: Stock Keeping Unit. A seller's unique identifier for a specific product variant.

Snowflake
: Cloud data warehouse used as the primary storage and query engine for input and output tables.

SQL
: Structured Query Language.

Tier
: One of nine signal-availability categories assigned to each seed by the Step 2 orchestrator. Each tier maps to a specific estimation pipeline.

VC / Vendor Central
: Amazon Vendor Central. Amazon's portal for manufacturers and brand owners, providing access to first-party (sourcing view) and total (manufacturing view) sales actuals.

vCPU
: Virtual CPU. The unit of compute allocated to a Fargate task, used as the basis for the energy and carbon estimates in Section 8.4.

WMAPE Weighted Mean Absolute Percentage Error. The primary evaluation metric for market share prediction quality, weighting errors by actual market share to reduce the influence of long-tail products.

XGBoost Extreme Gradient Boosting. The gradient-boosted tree library used for Pipelines 3 and 4.

Chapter 1

Introduction

1.1 Background and Context

E-commerce has grown into one of the largest and fastest-expanding sectors in the global economy. In 2024, worldwide e-commerce sales surpassed an estimated $6 trillion, representing more than 20% of total retail sales, with projections indicating continued year-over-year growth [1]. This expansion has fundamentally reshaped how consumers discover, evaluate, and purchase products: online retail ecosystems now host thousands of competing items within each category, turning visibility and discoverability into key competitive advantages for brands and manufacturers.

This environment has accelerated the rise of *retail media* — advertising and marketing strategies embedded directly within retailer platforms. Retail media influences consumers at the point of purchase through sponsored placements, keyword bidding, and promotional banners, making it one of the most measurable and high-intent marketing channels available to brands today.

Within this landscape, the concept of the *digital shelf* has become essential. The digital shelf encompasses all elements that determine how a product is presented and perceived on an e-commerce platform: search ranking, price, promotional status, inventory availability, customer ratings, and reviews. Tracking the digital shelf enables brands to monitor competitive positioning, diagnose visibility issues, and evaluate the effectiveness of retail media campaigns.

As e-commerce matures, brands and retailers invest heavily in analytics platforms capable of tracking product performance in near real time. Key performance indicators (KPIs) such as units sold, revenue, and market share are central to this analysis. Estimating these KPIs accurately remains challenging, however, due to the high dimensionality of digital shelf data, the frequency with which it changes, the heterogeneity of available signals, and the structural lack of ground-truth sales figures in most retail environments. Overcoming these challenges while maintaining a system that operates reliably at scale is the central engineering problem this thesis addresses.

1.2 The Company: Shalion

1.2.1 Positioning and Product Portfolio

This thesis was developed within Shalion, a technology company headquartered in Barcelona and founded in 2019. Shalion specializes in digital-channel data analytics, providing brands with what the company calls "extended intelligence" for digital shelf decision-making. The platform currently operates in over 85 countries, monitors more than 2,000 retailers worldwide, and processes millions of data points daily. Its technical foundation relies on the automated capture of publicly visible data on retailer websites — product titles, prices, stock status, Best Seller Rankings (BSR), and review counts — without requiring privileged API access or direct retailer integrations. The company serves global brands such as PepsiCo, Danone, Nestlé, Heineken, Diageo, and Mattel, among others.

Shalion organizes its offering around four commercial products, each delivered through an AI-driven interface called *Maestro* — an agentic recommendation engine that combines semantic search, adaptive reasoning, and narrative generation to translate complex digital shelf data into actionable insights:

Digital Shelf Maestro (DSM). Shalion's core digital shelf analytics product. It monitors full product categories daily across retailers, providing visibility into organic rankings, content compliance, pricing, promotions, availability, and share of voice. DSM is the primary product for brands seeking a comprehensive view of their digital shelf performance.

Retail Media Maestro (RMM). A cross-retailer retail media intelligence product primarily aimed at agencies. It tracks more than 200,000 keywords daily across the four major US retailers — Amazon, Walmart, Target, and Kroger — mapping keyword territory, share of voice, and competitive dynamics from the shopper's perspective. Market share estimation is a core component of RMM, providing BSR-based market share by brand and product without panel data lag.

Amazon Shelf Maestro (ASM). A specialized product tailored to Amazon's ecosystem, monitoring Buy Box ownership, Prime eligibility status, reviews, ratings, and advertising placements through a single Amazon-specific scorecard.

Outlet Media Distribution. A product focused on the food service sector, tracking product placement, pricing, and promotions across food delivery aggregators globally. This product does not involve market share estimation and falls outside the scope of this thesis.

The market share estimation service described in this thesis is the AI backend that powers the market share module within RMM and, selectively, within DSM and ASM. For RMM, market share is computed at scale for all tracked seeds across the four US retailers. For DSM and ASM, market share estimation is incorporated for specific clients who request it for particular categories. In both cases the estimation logic, data infrastructure, and modeling pipelines are identical — what differs is the set of seeds processed and the product surface through which results are delivered.

1.2.2 The Data Science and AI Team

The Data Science and AI team at Shalion operates as a cross-functional unit working in close collaboration with the Data Engineering team. In practice, the team's outputs are first integrated into Shalion's data infrastructure through a series of transformation and storage steps maintained by Data Engineering, and then consumed by the Product team for delivery to clients through the Maestro interface. The immediate stakeholders of the market share service are therefore the Data Engineering function and the data layer of the platform, with the Product team as the ultimate recipient of the outputs.

The team's activity centers on three analytical pillars: market share estimation, keyword traffic intelligence, and sales incrementality. Together these capabilities aim to connect digital shelf visibility to measurable commercial outcomes.

The author joined the team in February 2026 as an intern under a university internship agreement, taking on the role of lead developer for the market share project and remaining in this role until June 2026. The responsibilities spanned the full scope of the project: from diagnosing the scalability limitations of the existing system to redesigning the data flow, refactoring the codebase, training new models, and delivering a system running in production. The company supervisor was Magdalena Sztandarska; the academic tutor at UPC was Ramon Sangüesa Solé.

1.3 What Is Market Share Estimation?

Market share, in the digital shelf context, is the proportion of total category sales captured by a given product or brand within a defined retail environment *in a given week*. This thesis defines market share at the level of a *Product Listing Page* (PLP): the ranked list of products that a retailer displays for a given subcategory in a specific store.

The fundamental unit of estimation is a *seed* — a PLP of a specific subcategory in a specific store. A seed is therefore defined by the combination of subcategory and store; its product composition may vary by location (i.e. the same subcategory may surface different products depending on the delivery address or regional store). Market share is computed for a seed in a given location in a given week, based on the scraping carried out for that combination. For example, the "Soft Drinks" subcategory on Amazon tracked for a US ZIP code constitutes one seed in one location; the same subcategory on Walmart tracked for a different ZIP code is another seed-and-location combination.

For each seed and location in a given week, the system must estimate the sales volume of every product appearing in that PLP and then derive each product's share of the total. This task presents two structural difficulties. First, retailer platforms do not publish actual sales figures; the system must infer them from indirect signals such as BSR, units-sold indicators (NumberSells, NS), customer reviews and ratings, price, and promotional status. Second, the availability of these signals varies substantially across retailers and products. Not all products have a visible units-sold count; not all seeds correspond to Amazon, which is the only retailer providing first-party actuals through Vendor Central (VC); not all products have sufficient review history.

The estimation problem therefore has two regimes: estimating *absolute units* when strong demand signals are available, and estimating *relative market share* as a percentage within the

PLP universe when only weak or rank-based signals exist. Both regimes are addressed by a signal-adaptive routing mechanism called the *tier system*, described in detail in Chapter 6.

1.4 Motivation: From Prototype to Production

The starting point of this work is the system developed by Lluc Furriols Llimargas in his Bachelor's thesis [2], referred to throughout as System 0. System 0 demonstrated the feasibility of AI-driven market share estimation from digital shelf signals, deploying a modular pipeline for a limited set of clients and seeds. It was not, however, designed for company-wide production: its data retrieval strategy issued individual database queries for each seed-and-location combination and for each signal source separately, and it covered only a subset of the signal diversity present across Shalion's full client base.

Two business demands arrived concurrently and made scaling the system the immediate priority, shifting the focus of this thesis from model research to system engineering.

1.4.1 Priority 1 — The RMM Product at Scale

The launch of RMM required the market share service to track the most granular available subcategory of four major US retailers — Amazon, Walmart, Target, and Kroger — at full coverage. This expanded the number of active seeds to more than 20,000. When this seed volume was applied to System 0's data retrieval architecture, the weekly execution did not finish: it exceeded the 12-hour AWS connection limit and had to be aborted.

The root cause was query volume and queuing. System 0 issued individual database queries per seed and per signal source, generating more than 500,000 Snowflake queries per weekly execution at this scale. The Snowflake warehouse dedicated to the market share service could only process two to three queries simultaneously, so all remaining queries entered a queue. With hundreds of thousands of concurrent requests, the cumulative wait time in that queue — not the computation — made completion within any viable production window impossible. Solving this problem required a fundamental rethink of how data was retrieved and handed off to the modeling layer, and drove the architectural redesign that is the primary engineering contribution of this thesis (Chapter 5).

1.4.2 Priority 2 — Precise Per-Client Category Requests

In parallel, Shalion began receiving client requests for precise market share estimates in specific product categories. These requests exposed a second limitation of System 0: its models were not sufficiently accurate to meet client expectations for non-Amazon seeds and for seeds with limited signal availability. Improving estimation precision for these scenarios required the development of new modeling approaches. The two new gradient-boosted tree pipelines described in Chapter 6 were developed to address this demand.

1.5 Objectives

The two business priorities described above define the contribution of this thesis. Priority 1 — incorporating market share into RMM at scale — drives three of the six specific objectives: redesigning the data collection strategy, eliminating the query bottleneck, and building the revenue proxy needed to aggregate estimates across subcategories. Priority 2 — improving precision for specific client categories — drives the development of new modeling pipelines. Both priorities jointly motivate the codebase refactoring and orchestration work, and the monitoring and evaluation framework that supports the system.

The specific objectives are:

1. **Implement a new data-collection strategy for RMM**, adapting the scraping cadence to produce estimates aligned with natural calendar weeks (Monday to Sunday) and covering the most granular subcategory of the four major US retailers.

2. **Redesign the end-to-end data flow** to eliminate the per-seed query bottleneck and standardize the interface between data retrieval and modeling, making weekly production delivery feasible at more than 20,000 seeds.

3. **Develop and validate a revenue proxy** to approximate the revenue contribution of each product within a PLP, enabling future aggregation of subcategory-level market share estimates to higher category levels.

4. **Refactor the codebase and introduce tier-based orchestration**, replacing implicit data-availability logic with an explicit nine-tier routing system that assigns each seed to the most appropriate modeling pipeline based on the signals available for that seed and location.

5. **Design, train, and validate two new modeling pipelines** based on gradient-boosted trees, covering the non-Amazon and rank-only scenarios not addressed by System 0.

6. **Design a monitoring, evaluation, and quality assurance framework** that instruments the execution pipeline, measures prediction quality, and defines a strategy for systematic monitoring and testing as the system matures.

1.6 Scope

The market share estimation process at Shalion can be understood as a sequence of phases, from raw data collection to client-facing delivery. Figure 1.1 illustrates this end-to-end flow and marks which phases fall within the scope of this thesis and which do not.

The **web scraping** infrastructure that collects raw digital shelf signals from retailer websites, and the **raw data warehouse** layer that stores them, are maintained by a separate team and operate independently of the estimation service. They are out of scope.

The **data preparation** phase — where raw warehouse data is queried, filtered, and shaped into the weekly panel that feeds the models — is fully in scope, and its redesign is a core contribution of this thesis (Chapter 5). The **AI estimation pipelines** themselves, including the

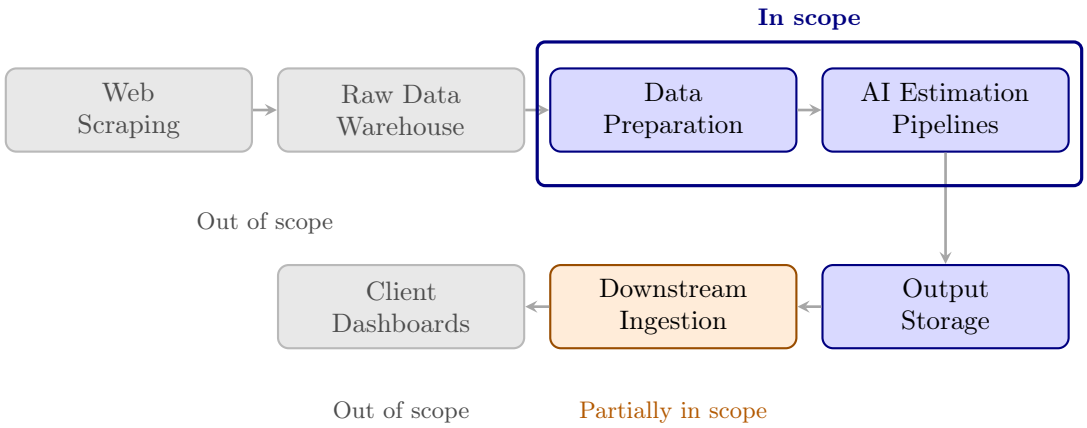

Figure 1.1: End-to-end market share estimation process at Shalion. Blue phases are fully within the scope of this thesis. The downstream ingestion step (orange) is described as context but not developed here. Grey phases are maintained by separate teams and outside scope.

tier-based orchestration logic, the five modeling pipelines, the revenue proxy, and the monitoring and evaluation framework, are equally in scope and constitute the primary technical contribution (Chapters 6 and 7).

The **output storage** layer, where weekly estimates are written and made available for downstream consumption, is in scope in the sense that the output contracts, data format, and storage strategy are designed as part of this work. The **downstream ingestion** step that reads those outputs and loads them into the tables used by the product is described for context but is not implemented as part of this thesis. The **client-facing dashboards** through which Shalion's clients consume market share estimates are entirely out of scope.

1.7 Methodology and Work Plan

1.7.1 Working Methodology

The Data Science and AI team at Shalion operates under an Agile methodology supported by a Kanban-style workflow for task management and prioritization. The team holds daily 15-minute stand-up meetings (Dailys) to review the previous day's progress, plan the current day's tasks, and surface any blockers. These sessions are complemented by weekly review meetings (Weeklies) where each active project is updated in greater depth and priorities are adjusted as needed. In addition to these team-level meetings, the author held regular checkpoints with both the company supervisor (Magdalena Sztandarska) and the academic tutor (Ramon Sangüesa Solé) throughout the internship period.

This combination of a fine-grained daily rhythm and periodic supervisory checkpoints proved particularly suited to the nature of the project: a system with multiple interdependent components where a blocker in one area — such as a memory overflow triggered by the new bulk query design — could redirect the plan for adjacent components within days. Work was decomposed into granular tasks, typically solvable within one or two workdays, which kept the schedule visible and allowed rapid reprioritization when technical obstacles arose.

1.7.2 Planning and Changes

The initial plan for this thesis centered on improving the modeling components of System 0: exploring new feature sets, testing alternative model architectures, and deepening the theoretical grounding of market share inference from weak signals. At that point the system operated with four scenario-based pipelines — three predictive AI pipelines plus an equal-share assortment fallback — and was serving a limited number of clients, with execution times that were acceptable at the existing seed volume.

This plan changed substantially when the RMM launch introduced the requirement to cover more than 20,000 seeds across four US retailers. The immediate consequence was that the weekly execution did not complete within the AWS 12-hour connection limit, and the project pivoted to system engineering as the critical path.

The first response was to reduce the number of input tables that the pipeline needed to query at runtime. This was achieved by modifying the DBT models: instead of having the pipeline query each dimension table separately for every seed-and-location execution, the dimension columns were embedded directly into the main BSR fact table at DBT construction time. The granularity change was from "one query per execution per table" to "one query at table-build time, zero queries per execution per dimension". This reduced total query volume substantially, but execution time remained too high. As an intermediate measure, the RMM client was split into batches of approximately 200 seeds, with each batch processed as a separate Fargate task (around 60 tasks per client). While this brought execution time down to 2.5–3 hours, it introduced a new cost concern: each Fargate task incurs compute cost, and 60 tasks per client per week becomes significant at scale. More importantly, the Snowflake queuing problem persisted, because multiple parallel tasks were still each issuing multiple queries, and the total queue pressure had only been partially relieved.

The next step was a more fundamental change: shifting query granularity entirely inside the execution. A single bulk SQL query per client batch now retrieves the complete weekly panel in one database round-trip. The Python layer then partitions the result by seed and location before dispatching each partition to the appropriate modeling pipeline. This brought the total number of database queries to one per client-and-batch, and reduced the ECS computation time for a single RMM batch (approximately 2,500 seeds) to roughly 15–20 minutes. Because batches run in parallel, the full weekly DAG — including the two DBT steps and all clients — completes in approximately one hour.

The bulk query design introduced a new constraint: loading the entire client panel into memory at once exceeded the memory available in the Fargate container. This was resolved by introducing an index-based batching mechanism that groups seeds into batches of approximately 2,500 and processes each batch as a separate orchestration task. The technical details and trade-offs of this approach are described in Chapter 5.

New modeling and infrastructure work was carried out in parallel with the architectural changes, paced by priorities and by which parts of the system were stable enough to build on at each stage. The two new pipelines, the revenue proxy, and the monitoring framework were developed progressively alongside the architecture iteration, rather than sequentially after it. A migration of the output storage to a modern table format with built-in versioning and schema

evolution was designed and implemented on a dedicated branch, but could not be merged into production due to a concurrent-write conflict that arises from the system's parallel execution model; this remains open work at the time of writing and is described in Chapter 5.

1.7.3 Budget and Economic Analysis

The project budget has two main components: personnel costs and infrastructure costs. Personnel covers the internship work and the supervision provided by the company and the university. On the infrastructure side, the system relies on Snowflake as the data warehouse and on a set of AWS services (primarily ECS/Fargate for compute and S3 for output storage); these costs are directly tied to the volume and efficiency of the weekly production run, making the architectural optimisations described in this thesis an ongoing source of operational savings. A detailed analysis of infrastructure cost across the architectural generations, including observed figures from production runs, is provided in **??**.

1.8 Structure of the Thesis

The chapters of this thesis follow a logic that moves from *context and foundations*, through *what the system does and how it works*, to *what it achieves and what it means*.

The first block — background, data, and the baseline system — establishes the territory. Before describing what was built, it is necessary to understand why digital shelf signals are structured the way they are, what the relevant prior work says about estimating demand from those signals, and what the starting point of this project looked like in concrete terms. The baseline chapter is not mere history: it defines the specific limitations that every subsequent decision was designed to overcome.

The second block — architecture and modeling — contains the core contributions. These two chapters are deliberately kept separate even though they interact closely. The architecture chapter addresses how data moves through the system: how it is retrieved, prepared, and delivered to the estimation layer efficiently at scale. The modeling chapter addresses what happens to that data once it arrives: how each seed is routed to an appropriate estimation strategy, how the models are structured, and how their outputs can be aggregated across the category hierarchy. Keeping these concerns separate makes it possible to reason about each independently, which reflects the actual structure of the codebase and the actual sequence of engineering decisions made during the project.

The third block — monitoring, ethics, and discussion — contextualizes the work. Monitoring and evaluation ask how well the system performs in production and how its health is observed. The ethics and sustainability chapter places the work within regulatory and environmental frames that any production AI system must address. The discussion chapter synthesizes the contributions, names the limitations honestly, and identifies the most productive directions for future work.

The structure is therefore not a tutorial that builds from simple to complex, nor a chronological account of the project. It is organized around the question of what a reader needs to understand in what order to follow the argument that this system is a meaningful engineering contribution to a real and difficult problem.

Chapter 2

Background and Literature Review

This chapter surveys the body of knowledge this thesis builds upon and situates the contribution within the existing literature. The work spans two domains that are seldom treated together: the applied problem of digital shelf analytics and market share estimation, and the engineering discipline of productionizing machine learning systems. The gap between them — and the absence of documented work on scaling inference-heavy ML pipelines in the digital retail domain to production at tens of thousands of inference units per run — is what this thesis addresses.

2.1 Digital Shelf Analytics and Market Share Estimation

2.1.1 From Panel Data to Scraped Signals

Traditional market share measurement relies on consumer panel data collected by specialist firms such as Nielsen IQ or Numerator. These panels provide high-quality, ground-truth sales data, but at the cost of latency (data typically arrives weeks after the period of interest), geographic granularity limits, and substantial cost that makes them inaccessible for continuous, category-wide competitive monitoring [3].

The rise of e-commerce created a complementary opportunity: retailers publish large amounts of publicly observable signals directly on their websites — product rankings, units-sold indicators, prices, promotional flags, reviews, and ratings. These signals are cheap to collect, updated frequently (often daily), and available for every product in every tracked category, including competitors. The trade-off is lower signal quality: the signals are indirect proxies for sales rather than direct measurements, they are noisy, and their availability varies substantially across retailers (Section 3.1.1). The research challenge is to extract reliable demand estimates from these imperfect but abundant proxies.

2.1.2 The Best Seller Ranking as a Sales Proxy

The Best Seller Ranking (BSR) is the most informative public signal available on Amazon and several other retailers. It directly reflects relative sales performance within a category and is updated frequently. The key empirical property of the BSR is that the rank-to-sales relationship within a category follows a power-law decay: the top-ranked product captures a disproportionately large share of category sales, and the distribution drops off rapidly with rank.

Chevalier and Goolsbee [4] provided an early quantification of this relationship in online book markets, demonstrating that a 10% improvement in rank is associated with a substantial increase in sales. Their work established the foundation for using rank as a demand proxy and showed that the rank-sales curve is stable enough to be modelled parametrically. Subsequent work confirmed the power-law structure across categories and retailers, making isotonic regression and log-linear models the natural methodological choices for rank-based sales estimation — choices that are reflected in Pipelines 1 and 2 of this thesis.

2.1.3 Review and Rating Signals

Customer reviews and ratings are the second family of widely available demand signals. Hu et al. [5] showed that review volume (not just sentiment) is positively correlated with sales, reflecting the dual role of reviews as both a consequence of demand (reviews accumulate as products sell) and a driver of future demand (high review counts build trust). Zhu and Zhang [6] further documented that the relationship between reviews and sales is moderated by product characteristics, with review signals more predictive for less familiar or niche products.

For the purposes of market share estimation, this thesis uses review and rating signals in two ways: as features in the XGBoost models (Pipelines 3 and 4) and as the basis for the revenue proxy that enables cross-category aggregation (Section 6.4). The latter use — weekly review deltas as a proxy for subcategory revenue share — is a novel contribution of this work, validated empirically in Section 6.4.5.

2.1.4 Market Share Estimation under Signal Scarcity

A key challenge not adequately addressed in the existing literature is the estimation of market share when direct demand signals (BSR, units-sold, actuals) are unavailable or unreliable. Most prior work focuses on Amazon or similarly data-rich environments. The extension to retailers that provide only price, image, and title — with no rank or engagement signals — requires a fundamentally different approach.

The tier-based routing system developed in this thesis (Section 6.2) addresses this gap by adapting the estimation strategy to the signal environment of each seed, rather than assuming a uniform signal landscape. The use of pseudo-labels from data-rich executions (Pipeline 1 outputs) to train models for data-poor contexts (Pipelines 3 and 4) is a form of semi-supervised learning that has theoretical grounding in the knowledge distillation and teacher-student literature [7], applied here to the cross-retailer signal transfer problem.

2.2 Productionizing Machine Learning Systems

2.2.1 The Gap Between Prototype and Production

Sculley et al. [8] identified the core challenge of industrial ML systems: while model code typically represents a small fraction of total system complexity, the surrounding infrastructure — data collection, feature engineering, monitoring, serving, and feedback loops — constitutes the majority of engineering effort. Their concept of "hidden technical debt" in ML systems is directly

relevant to the baseline described in Chapter 4: a system that works well for a small number of seeds in a controlled environment but accumulates structural debt (per-seed queries, hardcoded configurations, unversioned outputs) that becomes prohibitive at scale.

The field of Machine Learning Operations (MLOps) has developed a set of practices to address this debt [8]. Paleyes et al. [9] surveyed the challenges of deploying ML systems in production across industry, identifying data management, model monitoring, and reproducibility as the most commonly reported failure modes — all three of which manifested in the baseline system described in Chapter 4. The growing body of work on production ML systems emphasises that deployment is not the end of the engineering effort but the beginning of a new phase: the system must be observed, maintained, and iterated upon continuously.

2.2.2 Data-Centric AI and Feature Quality

A parallel strand of work has argued that improving data quality often yields larger gains than improving model architectures for real-world tasks [10]. This "data-centric AI" perspective is directly relevant to this thesis: the two most impactful improvements to estimation quality in this project were not model changes but data changes — the redesign of the BSR input table to embed dimension attributes (eliminating per-seed dimension lookups), and the introduction of the manufacturing-view actuals from Vendor Central (extending 1P coverage to include 3P sales). In both cases, better data at the input boundary improved all downstream models simultaneously.

The concept of a feature store — a centralised repository for pre-computed, versioned, and reusable feature sets shared across models [11] — reflects the same principle applied at scale. The bulk query design in this thesis realises a lightweight version of this pattern: the SQL CTE chain computes all features that any pipeline may need (rank signals, historical windows, Amazon-specific joins) in a single pass over the warehouse, so that each Pipeline worker receives a complete, pre-joined feature set without issuing additional queries.

2.2.3 Pipeline Orchestration at Scale

Apache Airflow has become the standard for orchestrating multi-step ML pipelines in production environments. Its DAG-based model, dynamic task generation, and integration with cloud compute services (including ECS and Fargate) make it suitable for the kind of weekly batch inference described in this thesis. The two-level DAG design (Section 5.3) — a parent DAG for coordination and a child DAG for parallel Fargate execution — follows established patterns for scaling batch inference workloads while maintaining clear separation of concerns. Kleppmann [12] articulates the underlying principle: reliable large-scale data systems decompose into small, stateless, independently deployable units connected by well-defined interfaces, which is precisely the architecture this thesis builds toward.

2.2.4 Modern Data Formats: from Parquet to Iceberg

The choice of output storage format has significant operational consequences in production ML systems. Apache Parquet provides efficient columnar storage but requires manual versioning, does not support atomic writes, and depends on external refresh mechanisms to make new data

queryable. Apache Iceberg [13] addresses these limitations with a metadata layer that tracks every write atomically, enables time-travel queries over the table history, and supports schema evolution without external coordination. The tension between Iceberg's atomicity guarantees and the parallel write model required for throughput at scale — described in Section 5.5 — is a concrete instance of a broader challenge in distributed ML systems: how to maintain consistency guarantees when inference is parallelised across many independent workers writing to shared storage.

2.2.5 Gradient-Boosted Trees for Tabular Data

XGBoost [14] remains the dominant approach for tabular prediction tasks in industrial settings, outperforming deep learning alternatives on most structured datasets where the number of observations is moderate and feature engineering is meaningful [15]. Its efficiency, regularisation, and compatibility with group-aware cross-validation make it particularly suitable for the estimation task in this thesis, where executions (groups of products within a single PLP) must be kept separate during training to prevent leakage.

Optuna [16] provides a principled framework for hyperparameter optimisation via tree-structured Parzen estimation (TPE), which is more sample-efficient than grid or random search. Used in conjunction with GroupKFold cross-validation, it enables robust model selection even with a moderate number of training executions.

2.3 Gap Identification and Positioning of This Thesis

The intersection of digital shelf analytics and production ML engineering is largely absent from the academic literature. Prior work on rank-based sales estimation focuses on modelling quality in controlled settings, without addressing the operational challenges of running such models weekly at tens of thousands of inference units under real-time production constraints. Conversely, the MLOps literature discusses system patterns in generic terms without addressing the specific data characteristics of digital shelf signals (heterogeneous availability, power-law distributions, semi-supervised training).

This thesis occupies precisely that intersection. The baseline described in Chapter 4 represents the state of the art in the domain at the start of the project: a modular, scenario-based pipeline for market share estimation, deployed for a small set of seeds, demonstrating that the modelling approach is sound. The contribution of this work is to carry that approach from prototype to production at full scale, resolving the architectural, operational, and modelling gaps that the baseline exposed when confronted with RMM requirements.

Chapter 3

Data

This chapter describes the data that underpins the market share estimation service: where it comes from, how it is collected, and the structural properties that shape every modeling decision in later chapters. Because the system infers sales from indirect signals rather than observing them directly, a precise understanding of the data — its sources, its heterogeneity across retailers, and its limitations — is a prerequisite for the architecture and modeling work that follows.

3.1 Data Sources

The estimation service combines two fundamentally different classes of data: public signals scraped from retailer websites, available for every tracked product, and confidential first-party data, available only for a small subset of products whose brands have granted access.

3.1.1 Public Signals: the Product Listing Page and Product Detail Page

The primary public data source is the **Product Listing Page (PLP)**: the ranked grid of products that a retailer returns for a given subcategory query. Shalion's scraping infrastructure captures, for each product on the PLP, the publicly visible attributes that the retailer chooses to display. Figure 3.1 shows a representative Amazon PLP with the key signals annotated.

The key signals extracted from the PLP are:

Best Seller Ranking (BSR). The product's position within the category when the PLP is sorted by best sellers. The BSR is the single most informative public signal, as it directly reflects relative sales performance within the category.

NumberSells (NS). A units-sold indicator displayed by some retailers as a coarse monthly counter (e.g., "30K+ bought in past month"). It provides an approximate magnitude of absolute demand, but only for the products that display it and only at bucket resolution.

Customer reviews and ratings. The total review count and average star rating, which serve as proxies for cumulative demand and product engagement over time.

Price, promotions, and availability. On-page commercial attributes used to convert estimated units into revenue estimates and to detect promotional spikes.

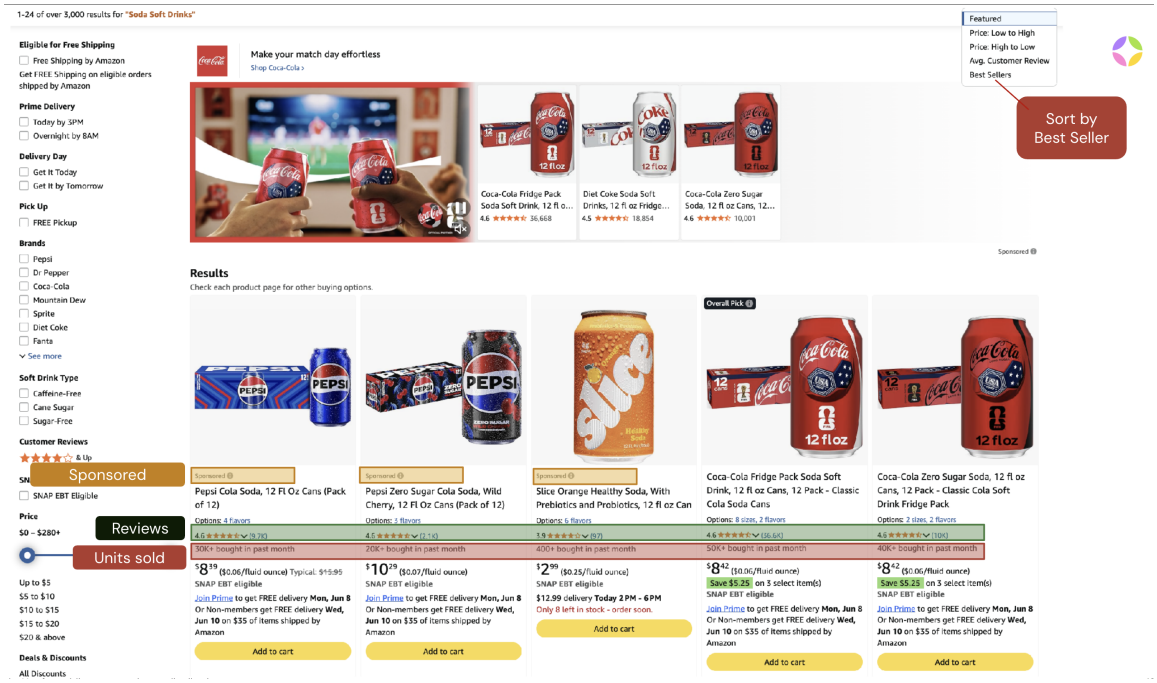

Figure 3.1: An Amazon Product Listing Page sorted by Best Sellers. Each product card exposes a different combination of signals: organic rank (position when sorted by Best Sellers), customer reviews and ratings, the units-sold indicator (e.g. "30K+ bought in past month"), price, and a sponsored/organic flag. Some products appear as sponsored placements that do not reflect organic demand.

Sponsored/organic flag. Whether the placement is an organic result or a paid advertising slot, used to filter out sponsored duplicates (see Section 3.3.1).

A critical property of PLP data is that **not every retailer exposes the same signals**. Figure 3.2 contrasts product cards from three retailers. Amazon US displays units-sold indicators, ratings, and prices; Target US displays ratings and prices but no units-sold counter; Carrefour ES displays only price and seller information, with no engagement signals at all. This heterogeneity directly motivates the tier-based routing system described in Chapter 6.

The Product Detail Page and 1P/3P prediction. In addition to PLPs, the system also extracts data from the **Product Detail Page (PDP)** — the individual product page that a shopper lands on after clicking a listing. The PDP exposes two signals not available on the PLP: **product variants** (colour, size, or pack-size options) and the **Buy Box**, which indicates which seller is currently winning the right to sell the product when a customer clicks "Add to Cart". Figure 3.3 shows a representative Amazon PDP with both signals annotated.

Variants are extracted but not currently modelled. The Buy Box is central to first-party vs. third-party prediction. In Amazon's marketplace, a product can be sold either directly by Amazon Retail (**first-party, 1P**) or by an independent third-party seller (**3P**). The Buy Box tells us which channel is currently fulfilling the product. Unlike the PLP, which Shalion scrapes weekly, the PDP is scraped **24 times per day** (once per hour) for clients who require the most precise market share estimates (Pipeline 1, Chapter 6). With 24 hourly observations per day, the system can compute the percentage of time during the week when Amazon itself held the Buy Box — a reliable proxy for the fraction of sales that were 1P. This Buy Box share is then used in

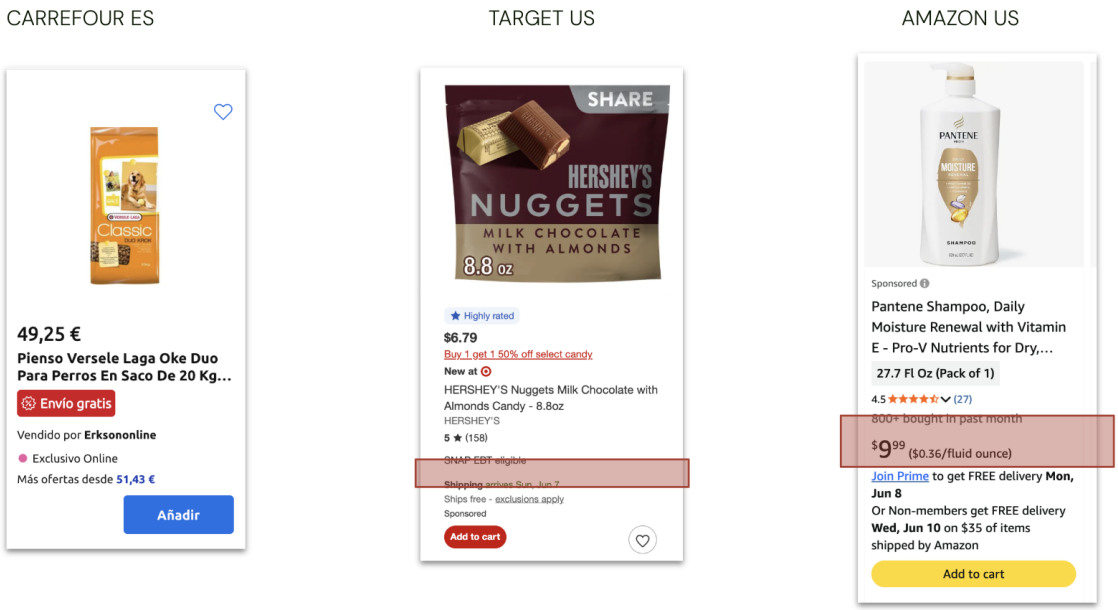

Figure 3.2: The same class of product across three retailers. Signal availability varies sharply: Amazon US exposes a units-sold counter, ratings, and price; Target US exposes ratings and price but no units-sold; Carrefour ES exposes only price and seller. The estimation system must produce a coherent market share estimate regardless of which signals are present.

Pipeline 1 to split the total predicted units into 1P and 3P components, providing clients with a granular view of the channel dynamics behind their market share.

3.1.2 Confidential Signals: Amazon Vendor Central Actuals

The second class of data is confidential sales data sourced from Amazon Vendor Central (VC), available only when a client grants Shalion access to their VC account. During this project two distinct views were integrated, which together provide substantially richer ground truth than was previously available:

Sourcing view. The view incorporated in System 0. It reports first-party (1P) sales: units shipped and revenue generated by Amazon Retail acting as the direct seller. This is the conventional understanding of "actuals" throughout this document.

Manufacturing view. A new addition introduced during this project. It reports *total* sales: first-party plus third-party (1P + 3P) combined, as Amazon records them from the manufacturer's perspective. Because it covers the entire market for a brand's products regardless of the selling channel, the manufacturing view allows a more precise calibration of total market share.

Both views are restricted to Amazon and require explicit credential sharing by the client. This creates a fundamental asymmetry in data availability. For clients tracked through DSM or ASM who opt in to VC integration, actuals are available for their specific products — typically a few hundred SKUs per client. For RMM, where the goal is to compute market share across all subcategories of the four major US retailers (Amazon, Walmart, Target, Kroger), there are no actuals at all: RMM seeds cover the full competitive universe of each category, not a single brand's catalogue, and no client holds VC credentials for every product in a category.

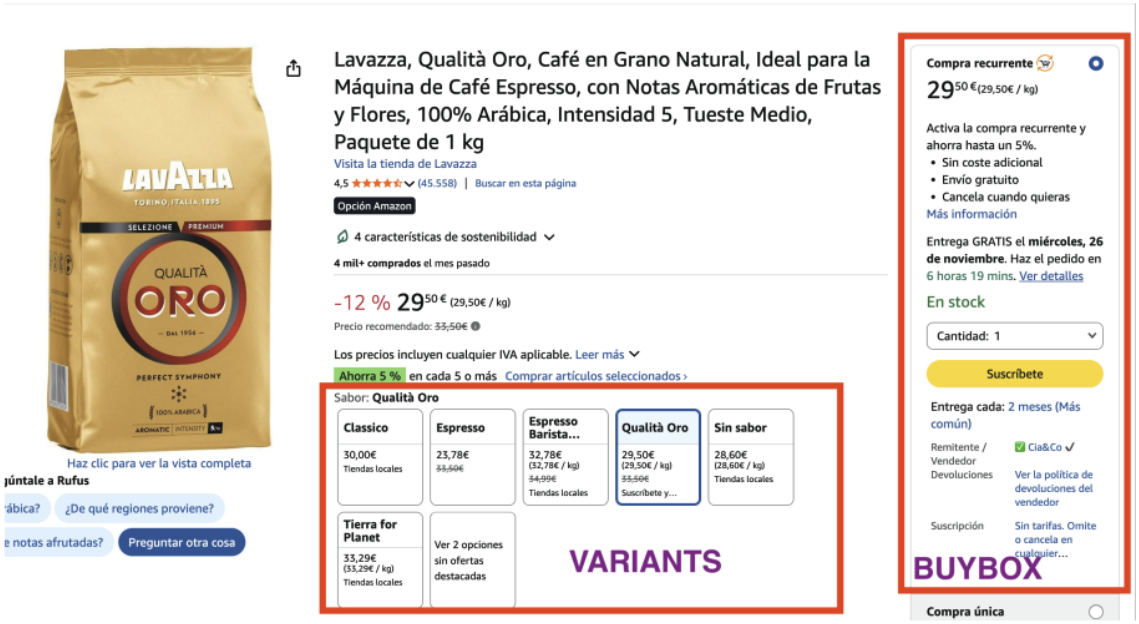

Figure 3.3: An Amazon Product Detail Page (Lavazza coffee), annotating the two signals extracted by Shalion. The **Variants** panel (red box, left) lists all available product variations (flavour, format, size), which are recorded but not currently modelled. The **Buy Box** (red box, right) shows the winning seller for the default "Add to Cart" action — here a third-party (3P) seller. When Amazon Retail wins the Buy Box, the product is classified as first-party (1P). Scraping the PDP hourly allows the system to estimate the weekly fraction of time each channel held the Buy Box, which drives the 1P/3P prediction in Pipeline 1.

To illustrate the scarcity: if Shalion has five VC-integrated clients, each with roughly 100 products, the system has access to approximately 500 products with ground-truth sales data. The remaining tens of thousands of products estimated weekly — and every single RMM seed — must be inferred entirely from public signals. Actuals are thus extremely valuable for the pipelines and model training where they are available, but they are the exception, not the rule.

3.2 New Data-Collection and Scraping Strategy

The expansion to RMM created the need for a revised data-collection schedule. System 0 scraped on **Wednesdays** and estimated the period from the previous Wednesday to the following Tuesday — a non-natural weekly boundary that did not align with standard commercial reporting calendars. The new strategy aligns the entire process with natural calendar weeks, so that every estimate corresponds to a clean Monday-to-Sunday window. Table 3.1 summarizes the convention.

Table 3.1: The new date and scraping convention for weekly market share estimation.

Element	Day	Role
Estimated period start	previous Monday	First day of the estimated week
Estimated period end	previous Sunday	Last day of the estimated week
PLP/BSR extraction (DATE)	Monday	Snapshot used for inference
DAG execution day	Friday	Weekly production run

The PLP snapshot is taken on the Monday immediately following the estimated week, capturing

the state of the catalogue at the close of that period. The production DAG then runs on the following Friday — four to five days later — for a reason directly tied to the Vendor Central actuals pipeline. Amazon Vendor Central data does not settle instantaneously: it typically takes three to four days for sales data from a given week to be fully populated and available through the VC API. Running the DAG on Friday therefore ensures that any actuals for the just-closed week have had sufficient time to arrive, so that Pipeline 1 (which calibrates against VC actuals; see Chapter 6) always operates on complete data rather than partially ingested figures.

Because scraped PLP data is not always available for the exact target date — a scraper may fail, or a retailer may not have been crawled on that precise Monday — the strategy also incorporates a **date tolerance**. When no snapshot exists for the target date, the system searches outward within a configurable window (governed by parameters `N_DAYS_BACK` and `N_DAYS_FORTH`), selecting the closest available execution. This tolerance makes the weekly run robust to isolated scraping gaps without compromising the alignment to calendar weeks.

3.3 Exploratory Analysis: Amazon-Specific Findings

Before redesigning the system, an exploratory analysis of production data revealed two structural properties of the PLP data with direct consequences for both data preparation and modeling. Both findings are most pronounced on Amazon, the richest and most complex retailer in the catalogue.

3.3.1 Sponsored Placements Must Be Excluded

Amazon PLPs interleave *organic* results — products ranked by genuine sales performance — with *sponsored* placements — paid advertising slots purchased by brands. These two result types look visually identical to a shopper but have fundamentally different meanings for demand inference: an organic rank reflects real consumer preference, whereas a sponsored position reflects advertising budget. As Fig. 3.4 shows, sponsored products are typically only identified by a small "Sponsored" label on the product card.

A dedicated study analysed the complete PLP extractions for one client across eight category seeds on a single date, comparing organic and sponsored rows. The study revealed three key structural facts about how products appear across the two types:

1. A product appearing as **organic** always appears **exactly once**, at its true BSR rank. This is the only rank meaningful for demand estimation.

2. A product appearing as **both organic and sponsored** appears **once organically** (at its true BSR rank) and **many times as sponsored**, one occurrence for each keyword or placement for which the brand is bidding. The organic rank is the correct one to use; the sponsored occurrences must be discarded.

3. Products appearing **only as sponsored** — with no organic rank in the category at all — are so few that they can be safely ignored: they do not belong to the organic category universe and including them would introduce noise without meaningful signal.

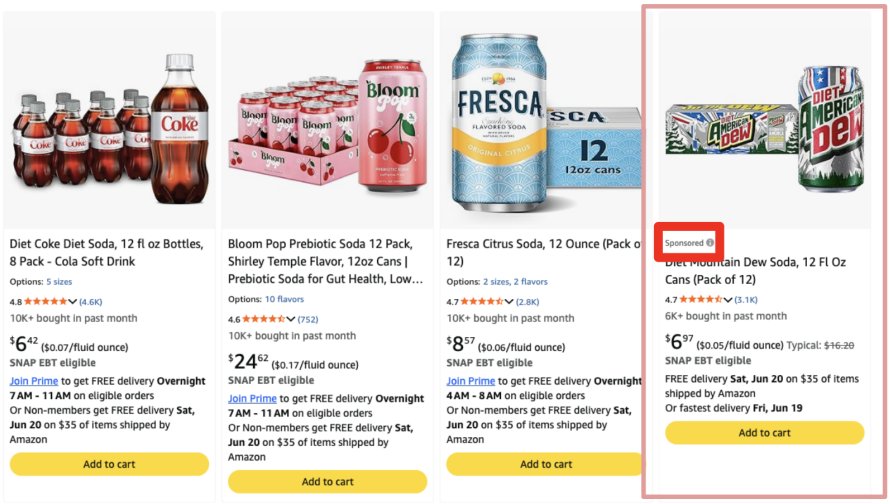

Figure 3.4: A sponsored product appearing in an Amazon PLP. The "Sponsored" label (highlighted) is the only visible indicator distinguishing an advertising placement from an organic result. Without filtering, sponsored rows would introduce duplicate records at misleading rank positions for products that also have a genuine organic rank.

Table 3.2: Distribution of products across organic and sponsored presence for each analysed category seed. "Organic only" products have a BSR rank and no sponsored duplicate; "Both" products appear once organically and one or more times as sponsored; "Sponsored only" products have no organic rank in this category.

Seed	Unique products	Organic only	Both	Sponsored only
Seed 1	968	951	9	7
Seed 2	1038	968	37	17
Seed 3	967	948	12	4
Seed 4	970	945	15	9
Seed 5	972	943	17	9
Seed 6	979	945	15	14
Seed 7	52	47	5	0
Seed 8	1938	1897	23	13
Total	**7884**	**7644**	**133**	**73**

Table 3.2 quantifies these patterns across the eight seeds.

The data confirm all three structural facts. Of 7,884 unique products, 7,644 (97.0%) appear only organically; 133 (1.7%) appear both organically and as sponsored; and just 73 (0.9%) appear only as sponsored. The "both" group — products with a true BSR rank that also appear sponsored — is the critical one: including their sponsored rows would introduce duplicate records at misleading positions. A product with an organic rank of 638, for instance, may appear sponsored at positions as high as 14 or as low as 1,276 across its various paid placements; mixing these with the true organic rank would significantly distort any rank-based sales estimate.

The conclusion is unambiguous: the data preparation layer must retain **only the organic result for each product**, discarding all sponsored rows. Products in the "sponsored only" group are excluded from estimation entirely, as they have no meaningful organic rank signal.

3.3.2 Sectors, Categories and Subcategories

The second finding concerns how category hierarchies are structured on Amazon and why this structure determines the correct estimation strategy for RMM. A study of more than 2.8 million product rows across four weekly executions analysed the hierarchical category paths encoded for each product (for example, *Home* > *Pet Products* > *Dog Supplies* > *Dog Food*).

The RMM estimation strategy. For RMM, market share is estimated at the level of the *last and most granular subcategory* — the deepest leaf in the category tree for each store. Figure 3.5 shows why this is the correct choice and what the data reveals about product membership across hierarchy levels. Two key observations motivate the strategy:

1. **Leaf subcategories contain all products.** Every product present in a retailer's catalogue appears in at least one leaf subcategory PLP. If a product also appears in a parent category PLP, it is always traceable to at least one of that parent's subcategories.

2. **Category PLPs are subsets, not supersets.** Scraping a parent category PLP does not give access to all the products in that category. The parent listing is always a strict subset of the union of its subcategories' products; many products appear exclusively in the leaf PLPs without surfacing at the category level at all.

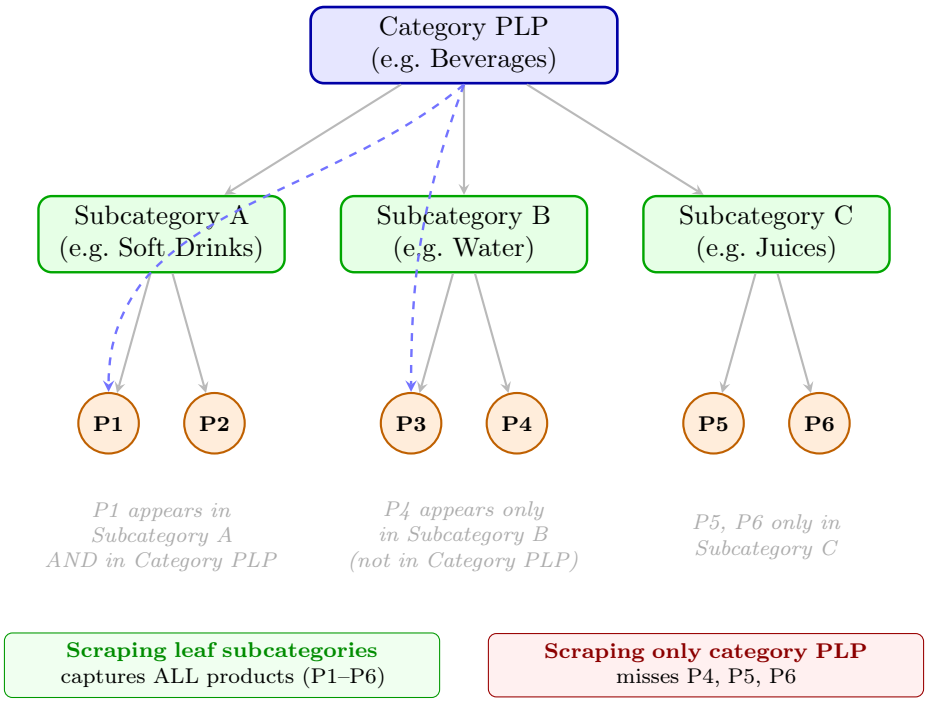

Figure 3.5: Category hierarchy and product membership. Products always appear in their leaf subcategory (green nodes). Some products additionally appear in the parent category PLP (dashed blue arrows), but the parent listing is never a superset of its children — many products appear only at subcategory level. Scraping only the category PLP would miss all products that are exclusively in subcategories (P4–P6 in this example). For RMM, Shalion must scrape the leaf subcategory PLPs to capture the complete product universe.

The conclusion for RMM is therefore: to capture *all* products in a category, Shalion must scrape the PLPs of the leaf subcategories — scraping only at the category level would miss a

significant fraction of the universe. Once leaf-level estimates are produced, they can then be aggregated upward using the revenue proxy (Section 6.4).

Quantitatively, the study found that 92.7% of product units (store × ASIN × date × pipeline) appear in exactly one category, while 7.3% appear in two or more — predominantly products listed in both a subcategory and its parent. Of the multi-category units whose parent category was present in the same execution, approximately 44% also appeared explicitly in the parent category listing, while the remaining 56% appeared only in the subcategory. No cases were found of products appearing at the parent level without a subcategory assignment.

This validates the RMM strategy: leaf subcategories are the atomic unit of estimation, and market share at any higher level can be obtained by aggregation. However, the 44% overlap means that naive summation of leaf estimates would double-count a subset of products, motivating the principled revenue-proxy aggregation developed in Section 6.4.

3.4 Data Limitations and Challenges

The preceding sections expose the central challenges the estimation system must contend with:

Ground-truth scarcity. Vendor Central actuals exist only for products of VC-integrated clients, covering a few hundred SKUs at most, and exclusively on Amazon. The system must therefore estimate sales for the vast majority of products with no direct supervision, and model evaluation is necessarily restricted to the data-rich subset. RMM seeds, by construction, never have actuals.

Signal heterogeneity. The set of available signals varies sharply across retailers and even across products within the same PLP. A model calibrated on the Amazon signal profile (BSR, NS, reviews, VC actuals) cannot be applied directly to a Carrefour product that exposes only price. This heterogeneity is the core motivation for tier-based routing (Chapter 6).

Noisy and inflated signals. Sponsored placements, when not filtered, inflate the apparent rank of advertised products. Coarse units-sold counters ("30K+ bought") compress wide ranges of true demand into a few buckets. Reviews accumulate over a product's entire lifetime rather than reflecting current-week sales.

Structural and temporal instability. Category trees are fragmented and products migrate across categories between executions. A product present one week may be absent the next, complicating consistent panel construction.

Partial category overlap. Because parent-category listings do not include all products present in their subcategories, and because a subset of subcategory products appear also at the parent level, naive summation of leaf estimates double-counts those products. A principled aggregation mechanism is required (Section 6.4).

Each of these challenges informs a specific design decision in the chapters that follow: ground-truth scarcity and signal heterogeneity drive the tier system and modeling (Chapter 6); noise and instability drive the data preparation and query refactoring (Chapter 5); and partial overlap drives the aggregation strategy (Section 6.4).

Chapter 4

The Baseline System (System 0)

This chapter describes the market share estimation system developed by Lluc Furriols Llimargas [2], referred to throughout this thesis as System 0. It constitutes the starting point from which all contributions in subsequent chapters depart. Understanding System 0 in detail — its design decisions, its strengths, and the constraints it operated under — is essential for contextualising the redesign work described in Chapters 5 and 6.

4.1 Overview

System 0 is a fully automated, end-to-end AI service for estimating product-level sales and market share in digital retail environments. It is designed to operate across multiple clients, retailers, and product categories, delivering consistent weekly estimates through a modular Python pipeline.

The system integrates heterogeneous digital shelf data — collected through Shalion's proprietary web scraping infrastructure and stored in the company's cloud data warehouse — into a weekly estimation workflow. For each product within a monitored category the pipeline produces: estimated total weekly units sold, the corresponding revenue (units multiplied by price), and the decomposition of total sales into first-party (1P, sold directly by the retailer) and third-party (3P, sold by third-party marketplace sellers) components.

The pipeline is orchestrated through a Directed Acyclic Graph (DAG) that parallelises execution across clients for a given reference date. Each execution is deployed as a containerised application that runs on a scheduled weekly basis, and is accompanied by a Streamlit monitoring application that allows inspection of individual predictions, aggregated performance metrics, and category-level sanity checks.

4.2 Data Inputs

System 0 consumes the data sources described in Chapter 3: the public PLP signals (Best Seller Ranking, NumberSells, reviews and ratings, price) and, where available, the confidential Vendor Central actuals that serve as ground truth. In addition to these, System 0 relies on one further input specific to its 1P/3P decomposition: the buybox ownership data scraped from each product's detail page. The buybox percentage records which seller holds the default purchase button and what fraction of the time, and is used downstream to split total predicted sales into

first-party and third-party components (Section 4.6). Product Detail Page scraping also supplies the child-ASIN structure, capturing product variants that do not appear in the PLP ranking directly.

All of these sources are merged, cleaned, and harmonised into a unified weekly panel per product — the input to the modelling stage. The preprocessing is handled by a set of transformation models maintained by Shalion's Data Engineering team; the estimation pipeline receives an analytics-ready dataset as its starting point.

4.3 System Architecture and Execution Flow

The execution model of System 0 is organised around three nested levels: a weekly Airflow DAG, a client-level *Pipeline Complete*, and a per-seed *Pipeline*. Figure 4.1 shows the full system data flow across the INPUT, STORAGE, COMPUTE, and CONSUMPTION domains. The COMPUTE domain — driven by the Airflow DAG on a weekly schedule — has three sequential steps: a DBT run that prepares the input tables from the data warehouse, an ECS/Fargate estimation stage that executes the market share computation, and a second DBT run that refreshes the mart tables from the output Parquet files written to S3.

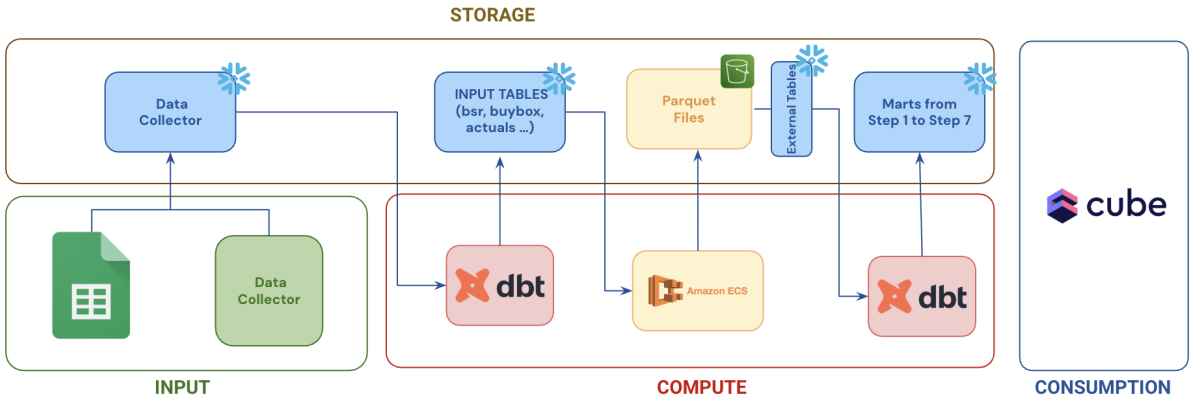

Figure 4.1: System 0 end-to-end data flow organised by domain. The INPUT domain supplies the scraping layer and a Google Sheets configuration file. The STORAGE domain holds the data warehouse input tables and the Parquet output files exposed via External Tables. The COMPUTE domain (Airflow DAG) drives three sequential steps: DBT input-table preparation, ECS/Fargate market share estimation, and DBT mart refresh. Results are delivered to the CONSUMPTION layer (dashboards).

DAG and configuration. The DAG is scheduled weekly and executes once per client and reference date. Its first task runs a set of DBT models that read from a Google Sheets configuration spreadsheet maintained by the operations team. This spreadsheet defined the universe of seeds to be estimated for each client: the list of subcategories, store identifiers, and location combinations that the system should process in a given week. The DBT step synchronised the spreadsheet into a structured configuration table in the data warehouse — `stg_marketshare_config` — which served as the ground truth for seed discovery throughout the System 0 lifecycle. This dependency on a manually maintained spreadsheet was one of the limitations addressed in the redesign: in System 2, the configuration table is eliminated entirely and the seed universe is derived automatically from the live BSR data present in the warehouse for each execution date.

After the configuration table was populated, the DBT step also prepared the BSR, actuals, and buybox tables from the raw warehouse data, making them available for downstream querying.

Pipeline Complete and the per-seed dispatch. Once the input tables were ready, the DAG triggered a single Fargate task per client, which executed *Pipeline Complete*. Pipeline Complete issued one query to the configuration table to retrieve the full list of seed-and-location combinations for that client and date. It then dispatched one parallel execution of *Pipeline* per combination, all sharing the same reference date and client context, using a thread-level worker pool.

Pipeline: data retrieval and computation per seed. Each *Pipeline* execution was entirely self-contained: it opened its own database connection and performed all data retrieval needed for a single seed-and-location pair before doing any computation.

Data retrieval was concentrated in two steps. Step 1 (data preparation) issued the bulk of the reads: a store-name lookup, a seed-count query, and a BSR query that, when the target date returned no data, triggered a retry loop probing progressively wider date windows (up to ±6 days, i.e. up to twelve additional attempts); plus queries for Vendor Central actuals, buybox ownership, parent-child product mappings, and store metadata. Steps 2 to 5 (feature engineering, modelling, inference, evaluation) performed no database I/O. At the other end of the pipeline, Step 6 (results) issued 4 further dimension-lookup queries to enrich the output contract with seed category, brand, client name, and store country — attributes needed by the dashboards that were not carried through the dataframe. In total, a single *Pipeline* invocation issued between 10 and 22 reads per seed (≈13 in typical conditions).

Steps 4 to 6 also wrote intermediate and final Parquet files to cloud storage. Once all parallel Pipeline runs completed, Pipeline Complete collected the results and signalled completion to the DAG. A final DAG task then ran a second set of DBT models that read the Parquet outputs and refreshed the downstream warehouse tables.

Figure 4.2 shows this internal structure and the database queries issued at each granularity level.

4.3.1 Query Volume at Scale

With between 10 and 22 database reads per seed-and-location combination (≈13 in typical conditions), the total query count across all clients exceeded **500,000 per weekly run** at the seed volumes required by RMM. The Snowflake warehouse dedicated to the market share service could only process two to three queries simultaneously, so all remaining queries entered a queue. The cumulative wait time — not computation — dominated total runtime: when the seed volume expanded to more than 20,000, the DAG exceeded the 12-hour AWS connection limit and did not complete, making weekly production delivery infeasible and motivating the architectural redesign described in Chapter 5.

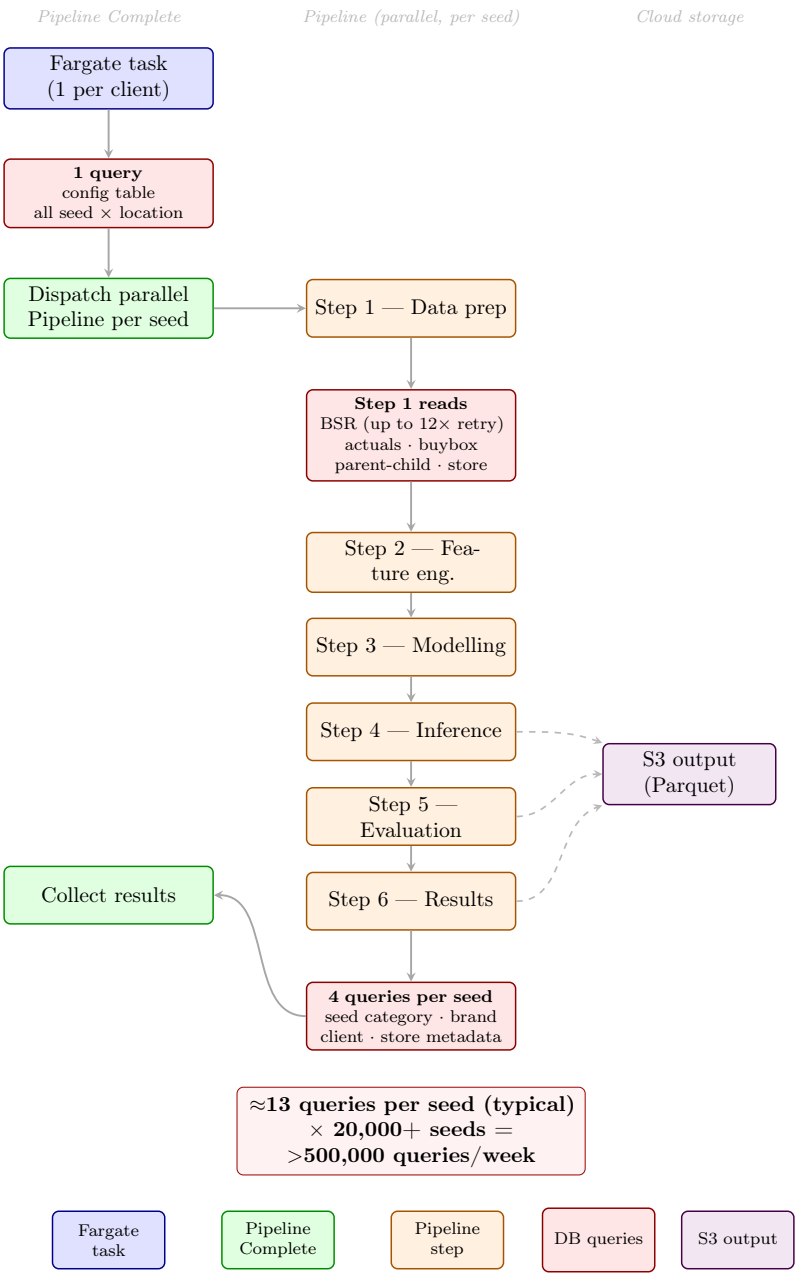

Figure 4.2: System 0 internal execution model inside the ECS/Fargate task. *Pipeline Complete* queries the configuration table for the full list of seed-and-location combinations and dispatches parallel *Pipeline* workers. Each worker issues its reads in Step 1 (BSR with up to twelve retry attempts on date-tolerance fallback, actuals, buybox, parent-child mapping, and store metadata) and 4 further dimension reads in Step 6 (seed category, brand, client, store country). With between 10 and 22 queries per seed (≈13 in typical conditions) and more than 20,000 seeds at RMM scale, total weekly query volume exceeds 500,000, overwhelming the shared Snowflake queue and causing the execution to time out.

4.4 Feature Engineering

System 0 constructs two families of derived features that are central to all pipelines.

4.4.1 Weekly NumberSells Estimation

The raw NS signal is a rolling monthly counter (e.g., "1K+ bought in past month"). A direct division by 4.345 (the average number of weeks per calendar month) gives a weekly baseline, but this assumes uniform demand throughout the month. To capture the actual weekly trend, the system computes the delta of the NS counter relative to its four-week moving average:

$$\Delta_{\text{NS}} = \frac{NS_t - \bar{NS}_{(t-4:t-1)}}{\bar{NS}_{(t-4:t-1)}} \tag{4.1}$$

The final weekly NS estimate combines the baseline with this momentum multiplier:

$$\widehat{NS}_{\text{weekly},t} = \frac{NS_t}{4.345} \times (1 + \Delta_{\text{NS}}) \tag{4.2}$$

This formulation assigns more weight to the current week when the monthly counter is growing rapidly, and less when it is stagnant — correctly weighting promotional spikes and demand troughs without requiring explicit promotional flags.

4.4.2 Temporal Momentum: the Inertia Score

To capture short-term product momentum, System 0 computes a delta percentage for each available signal s (Rank, NumberSells, Reviews, 1P actuals):

$$\Delta_s = \frac{x_{s,t} - \bar{x}_{s,(t-4:t-1)}}{\bar{x}_{s,(t-4:t-1)}} \times 100 \tag{4.3}$$

The Inertia Score is the arithmetic mean of the deltas of all signals available for a given product:

$$\text{Inertia} = \frac{1}{|F_{\text{avail}}|} \sum_{s \in F_{\text{avail}}} \Delta_s \tag{4.4}$$

where F_{avail} is the set of non-null features for that product-retailer combination. Missing signals are omitted and the denominator adjusts accordingly, making the score robust to varying signal availability across retailers.

The Inertia Score is applied as a post-hoc multiplicative adjustment to the rank-based model output, shifting individual product predictions toward their recent trajectory. This separation of concerns — stable global rank-sales relationship in the regression, local trend correction via inertia — is a design pattern carried forward into the new system.

4.5 Modelling Approach: Scenario-Based Orchestration

The central design decision in System 0 is the recognition that a single universal model cannot serve all data availability situations. An orchestrator module in Step 3 inspects the signals present

for each execution and routes it to the appropriate pipeline. Four scenarios are defined:

Table 4.1: Signal availability by scenario in System 0. ✓ = available, × = not available.

Signal	Scenario 1	Scenario 2	Scenario 3	Scenario 4
1P Actuals (Vendor Central)	✓	×	×	×
NumberSells (NS)	✓	✓	×	×
Best Seller Rank	✓	✓	✓	×
Ratings / Reviews	✓	✓	✓	×
Content fields	✓	✓	✓	✓

The orchestrator evaluates signals in priority order — actuals first, then NS, then rank, then fallback — and routes to the strongest applicable pipeline.

4.5.1 Pipeline 1 — Full Signals with Actuals

When VC actuals are available, Pipeline 1 calibrates the NS-based proxy against real sales using a log-linear model. The logarithmic deviation between the proxy and the actual is expressed as a function of product rank:

$$y_i = \log\left(\frac{\text{actual\_weekly}_i}{\widehat{NS}_{\text{weekly},i}}\right) = \beta_0 + \beta_1 \log(\text{rank}_i) + \varepsilon_i \tag{4.5}$$

This formulation captures the systematic underestimation of top-ranked products without directly predicting absolute sales from rank alone. Outliers are filtered using an IQR-based criterion before fitting. The resulting prediction is:

$$\widehat{\text{sales}}_i = e^{\hat{\beta}_0} \cdot \widehat{NS}_{\text{weekly},i} \cdot \text{rank}_i^{-\hat{\beta}_1} \tag{4.6}$$

Pipeline 1 serves as the ground-truth reference for the entire system: its predictions are used as pseudo-labels to train Pipeline 3.

4.5.2 Pipeline 2 — NumberSells Without Actuals

When VC actuals are absent but NS is present, Pipeline 2 estimates the rank-to-sales curve directly from the NS proxy. A log-linear model is fitted on the current week's data augmented with the four preceding weeks, giving each product up to five observations and making the estimated curve more stable:

$$\log(\widehat{NS}_{\text{weekly},i}) = \alpha + \beta \log(\text{rank}_i) + \varepsilon_i \tag{4.7}$$

The model learns a smoothed, rank-consistent baseline; short-term demand fluctuations are handled downstream by the Inertia Score. This separation prevents overfitting to transient weekly patterns.

4.5.3 Pipeline 3 — Rank Without NumberSells

When neither actuals nor NS are available but BSR and engagement signals exist, Pipeline 3 shifts from absolute sales estimation to relative market share estimation. Because the signal

environment is too sparse for per-execution calibration, it uses a *pre-trained* XGBoost regression model applied at inference time.

The training target is the log-transformed market share derived from Pipeline 1 executions (pseudo-labels). Features include product rank, review and rating signals scaled within-execution, and distribution-level statistics aggregated across the execution. The model is trained with group-aware cross-validation (GroupKFold by execution), with sample weights 1/rank to emphasise top-ranked products, and with Duan's smearing estimator [17] to correct the log-transformation retransformation bias.

During inference, the model predicts $\log(\widehat{MS})$; the prediction is exponentiated, multiplied by the smearing factor, and normalised to sum to one within each execution.

4.5.4 Pipeline 4 — Minimal Signal Fallback

When no rank, NS, or review signals are available, System 0 defaults to an equal market share fallback: each product receives $1/N$ of the total, where N is the number of products in the category. This pipeline prioritises continuity of service over precision.

4.6 Refinement, Output and Deployment

After the modelling step, a common refinement module handles price imputation (using a hierarchical strategy: recent weekly history, parent-ASIN mean, or category-level average), converts unit predictions to revenue estimates, and decomposes total predicted sales into 1P and 3P components using the buybox ownership percentage:

$$\text{1P Sales} = \widehat{\text{Total Sales}} \times \text{BuyboxShare} \tag{4.8}$$

$$\text{3P Sales} = \widehat{\text{Total Sales}} - \text{1P Sales} \tag{4.9}$$

A fixed fallback of 90% is applied when buybox data is missing. For Pipelines 3 and 4, absolute volumes are converted to percentages before output.

Outputs are written in two forms. Intermediate and final Parquet files are stored in cloud object storage, partitioned by date, run identifier, client, seed, and location. A set of dimension-enriched tables is also written directly to the data warehouse (Step 6 issues four dimension lookups for this enrichment: seed category, brand and manufacturer, client, and store country). These warehouse tables are the source consumed by the client-facing dashboards.

The system is deployed as a Docker container image and executed through the Airflow DAG, with continuous deployment: any commit to the repository triggers a new image build, which is pushed to the container registry and picked up by the next DAG run without manual intervention.

4.7 Evaluation and Baseline Performance

Evaluation in System 0 is constrained by ground-truth scarcity: performance can only be measured for the subset of seeds where VC actuals are available. Results should therefore be interpreted as

indicative of performance in data-rich conditions.

The evaluation is designed to operate at two complementary levels. At the *item level*, WMAPE and MAPE quantify how accurately the model predicts individual product sales. At the *aggregate level*, the Client Total Error measures the deviation between the sum of all predicted sales and the sum of actuals for a given client — the metric most directly relevant to business decision-making, as it answers whether the total market share estimate for a brand is correct regardless of product-level swaps.

Table 4.2 reports the performance figures extracted directly from the System 0 monitoring application [2]. Pipeline 4 is excluded from quantitative evaluation: its purpose is continuity of service in the absence of any predictive signal, not accuracy.

Table 4.2: Performance of System 0 across the three evaluated pipelines, extracted from the monitoring application [2]. Pipeline 2 appears in the interface as *isotonic / using midpoint*. Pipeline 4 is excluded as a continuity fallback. Pipeline 3 operates in relative market-share mode and its Client Total Error is therefore not directly comparable to Pipelines 1 and 2.

	Item-level (%)		Aggregate (%)
Pipeline	WMAPE	MAPE	Client Total Error
Pipeline 1 — actuals + NS	25.82	33.45	3.82
Pipeline 2 — NS, no actuals	31.45	49.36	21.64
Pipeline 3 — rank + reviews	54.64	56.45	44.91

Pipeline 1 achieves the strongest results. The Client Total Error of 3.82% confirms that product-level errors largely cancel out when aggregated to brand level, making the system commercially reliable for clients with Vendor Central access. The gap between WMAPE (25.82%) and MAPE (33.45%) is significant: the model is considerably more accurate on high-volume, top-ranked products than on the long tail — a desirable property, since high-volume products carry the greatest commercial weight.

Pipeline 2 degrades gracefully without actuals. The aggregate error rises to 21.64%, still within a range that supports competitive benchmarking. The MAPE of 49.36% is substantially higher than Pipeline 1, reflecting increased product-level noise when the NS proxy must substitute for direct calibration; nonetheless, errors partially cancel at aggregation.

Pipeline 3 produces the weakest figures — WMAPE 54.64%, MAPE 56.45%, Client Total Error 44.91% — reflecting the fundamental difficulty of inferring relative market share from rank and engagement signals alone. Unlike Pipelines 1 and 2, WMAPE and MAPE are nearly equal, indicating that errors are uniformly distributed across all products with no volume-based compensation at the top of the ranking. Despite this, Pipeline 3 retains directional value: it correctly orders competitors within a category and provides a relative view of market positioning in retailer environments where no stronger signal is available.

Performance is also non-uniform across clients and categories. The monitoring interface reveals that stable categories with consistent BSR patterns achieve near-zero aggregate errors on Pipeline 1, while volatile categories with frequent promotions show larger deviations. This observation motivates the Inertia Score described in Section 4.4.2, which partially mitigates short-term volatility by adjusting predictions toward each product's recent trajectory.

4.8 Limitations and Motivation for the Redesign

System 0 represents a genuine proof of concept for AI-driven market share estimation at Shalion. Its modular pipeline, scenario-based orchestration, containerised deployment, and monitoring interface provided a solid foundation. However, four structural limitations made it unsuitable for the scale and coverage requirements that emerged with the RMM launch.

Per-seed, per-source query architecture. With between 10 and 22 database reads per seed-and-location combination (≈13 in typical conditions), the system generated more than 500,000 queries per weekly run at RMM scale. The shared query queue introduced cumulative wait times that pushed total runtime beyond 14 hours: only the ECS compute task is bound by the 12-hour AWS connection limit, while the surrounding DBT tasks have no such limit, so the overall DAG could run longer even though the compute task itself was aborted at the 12-hour mark, as detailed in Section 4.3.1. Eliminating this bottleneck is the primary objective of the redesign in Chapter 5.

No coverage for non-Amazon signal profiles. The four-scenario structure was calibrated for the Amazon signal environment (BSR, NS, VC actuals). As shown in Section 3.1.1, the Walmart, Target, and Kroger seeds introduced by RMM expose markedly different signal sets — in particular, different null-rate profiles for reviews and ratings — that the existing scenario logic could not accommodate. Addressing this gap required the extended tier system described in Chapter 6.

No cross-level aggregation mechanism. System 0 estimated market share at seed level only. No mechanism existed for aggregating share estimates from the scraped subcategory level up to broader category hierarchies — a requirement made non-trivial by the partial category overlap documented in Section 3.3.2. This gap motivates the revenue proxy developed in Section 6.4.

Unversioned output storage. Outputs were written as plain Parquet files with no schema versioning, no rollback capability, and no distribution tracking over time. The designed migration to a versioned table format addresses this limitation and is described in Section 5.5.

Each of these limitations maps directly to a contribution of this thesis, making System 0 not merely a historical reference but an active design specification for the work that follows.

Chapter 5

System Architecture and Data Engineering

This chapter describes the redesign of the market share estimation service from System 0 (Chapter 4) into a production-grade system capable of running at the scale of RMM. It is the principal engineering contribution of this thesis. The chapter is organised around the lifecycle of a weekly execution: Section 5.1 gives the high-level picture of how data flows end to end and how the orchestration, the per-task code, and the storage layers fit together; Section 5.2 describes the development and production environments that allow this system to evolve safely; Section 5.3 details the Airflow orchestration; Section 5.4 details the code refactoring that eliminated the query bottleneck; and Section 5.5 covers the designed migration to a modern output table format.

5.1 Standardised End-to-End Data Flow

At the highest level, producing the weekly market share estimates is a pipeline that turns raw scraped data into dashboard-ready tables. Figure 5.1 organises this process around three domains that map to the ownership and technology boundaries in the system. Compared to System 0 (Chapter 4), the data flow has been significantly redesigned at every layer of the Compute domain: the configuration spreadsheet that drove seed discovery in System 0 has been eliminated, the set of DBT input tables has been reduced and enriched to remove per-seed dimension lookups, and the code executed inside the ECS tasks has been restructured to replace per-seed Snowflake queries with a single bulk retrieval, introduce a nine-tier orchestration system, and improve code organisation and parallelism.

The three domains. The **Input** domain supplies the raw material. The scraping layer runs continuously and stores scraped PLP extractions in the data warehouse. Amazon Vendor Central actuals are ingested separately for clients who have granted access. Both sources feed the data warehouse, which is the boundary of the Input domain. Unlike System 0, there is no longer a Google Sheets configuration spreadsheet as an input: the universe of seeds to process is determined dynamically from the BSR data present in the warehouse for each execution date.

The **Storage** domain holds every persistent artefact. It spans the data warehouse (for structured tables), S3 (for Parquet output contracts and external tables), and the mart tables that

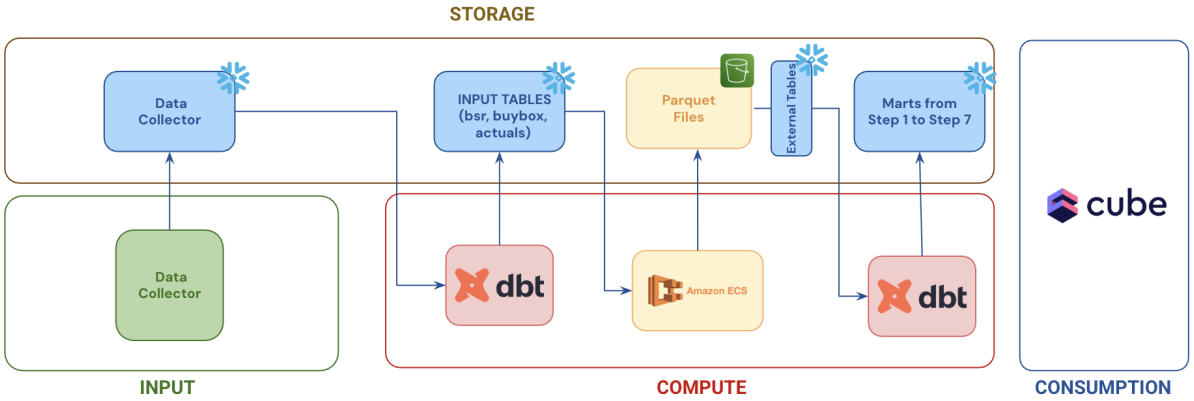

Figure 5.1: End-to-end market share data flow organised by domain (System 2). The **Input** domain supplies raw scraped data via the Data Collector and Amazon Vendor Central actuals. The **Storage** domain holds the data warehouse input tables (BSR, buybox, actuals), the Parquet output files from the estimation pipeline, an External Tables layer (S3 → Snowflake connector), and the mart tables consumed by dashboards. The **Compute** domain (Airflow DAG) drives three sequential steps — DBT input-table preparation, Amazon ECS/Fargate estimation (each Fargate task runs *Pipeline Complete* for one client-and-batch combination, which parallelises *Pipeline* per seed and location), and DBT mart refresh. The **Consumption** layer (Cube) delivers estimates to client-facing dashboards.

are the final product consumed by dashboards. The DBT input tables in this layer are now fewer and richer than in System 0: the main BSR table embeds dimension attributes (brand, category, store, batch index) that were previously retrieved by per-seed queries at runtime, eliminating a whole category of Snowflake round-trips.

The **Compute** domain contains all transformation logic and corresponds to the Airflow DAG described in Section 5.3. It has three sequential steps:

1. **DBT input-table preparation** (Airflow Task 1) transforms the raw warehouse data into estimation-ready tables: the BSR catalogue with dimension attributes and batch indices embedded, the buybox table, and the actuals table. This step also implicitly defines the seed universe for the week, replacing the manual configuration spreadsheet of System 0.

2. **Amazon ECS/Fargate computation** (Airflow child DAG, Task 3) performs the actual market share estimation. Each ECS task corresponds to one client-and-batch-index combination and executes *Pipeline Complete*, which issues a single bulk query and parallelises *Pipeline* over every seed-and-location in that batch. The key improvements over System 0 are: zero per-seed queries, a nine-tier orchestration replacing the four-scenario logic, and explicit memory management through batch indexing.

3. **DBT mart refresh** (Airflow Task 5) reads the S3 Parquet files through external tables and populates the mart tables in the data warehouse, making the estimates available to the client-facing dashboards.

How the code fits in. Within the ECS/Fargate computation step, the code is organised into two components with a clear separation of concerns. *Pipeline Complete* is responsible for all data access and parallelisation: it issues a single bulk query against the prepared input tables, receives the complete weekly panel as a dataframe, partitions it by seed-and-location, and dispatches

parallel workers. *Pipeline* receives one in-memory slice and performs the estimation for a single seed-and-location pair, with no further database access. This two-component design, and the refactoring that produced it across successive architectural decisions, is described in detail in Section 5.4.

5.2 Development and Production Environments

One of the early engineering decisions in this project was to establish a clear separation between development and production environments. In System 0 and the intermediate architecture, changes were tested and deployed from a single branch directly to production, with no isolated environment to validate new code before it affected live clients. As the system grew in complexity this became untenable. The solution is a two-environment model tied to the Git branching strategy.

5.2.1 Code and Branch Structure

The repository maintains two long-lived branches. The `main` branch represents the production system: all code on `main` is considered stable and runs weekly for live clients. The `develop` branch is the integration branch for new features: work is developed on feature branches, reviewed via pull request, and merged into `develop` first. Only after validation in the development environment is a merge from `develop` into `main` allowed.

A continuous integration workflow triggers on every push to either branch, builds a Docker container image, and pushes it to the container registry. Each branch pushes to a **separate registry repository**, so a broken development build never overwrites the production image.

5.2.2 Infrastructure Separation

The two environments are isolated at every layer of the stack:

Container images. Separate registry repositories per environment; the Fargate task for each environment pulls from its own registry.

Cloud storage. Output Parquet contracts are written to separate storage prefixes; a development run cannot overwrite production outputs.

Data warehouse schemas. DBT models exist in both environments under distinct schema prefixes. A development run reads and writes to development schemas, leaving production schemas untouched.

AWS credentials. Each environment uses dedicated credentials, preventing accidental cross-environment writes.

In practice, a developer can push a feature branch to `develop`, trigger a full test run on the development Fargate cluster against real data, and inspect outputs in the development storage prefix — with no risk to the live weekly production run. The Iceberg migration (Section 5.5) was developed and tested entirely in this development environment.

5.3 The Airflow DAG and Execution Architecture

As introduced in Section 5.1, the weekly execution is orchestrated by a two-level Airflow DAG structure. A parent DAG handles data preparation, client discovery, and post-processing; a child DAG carries out the market share computation on Fargate. Figure 5.2 shows the complete task graph.

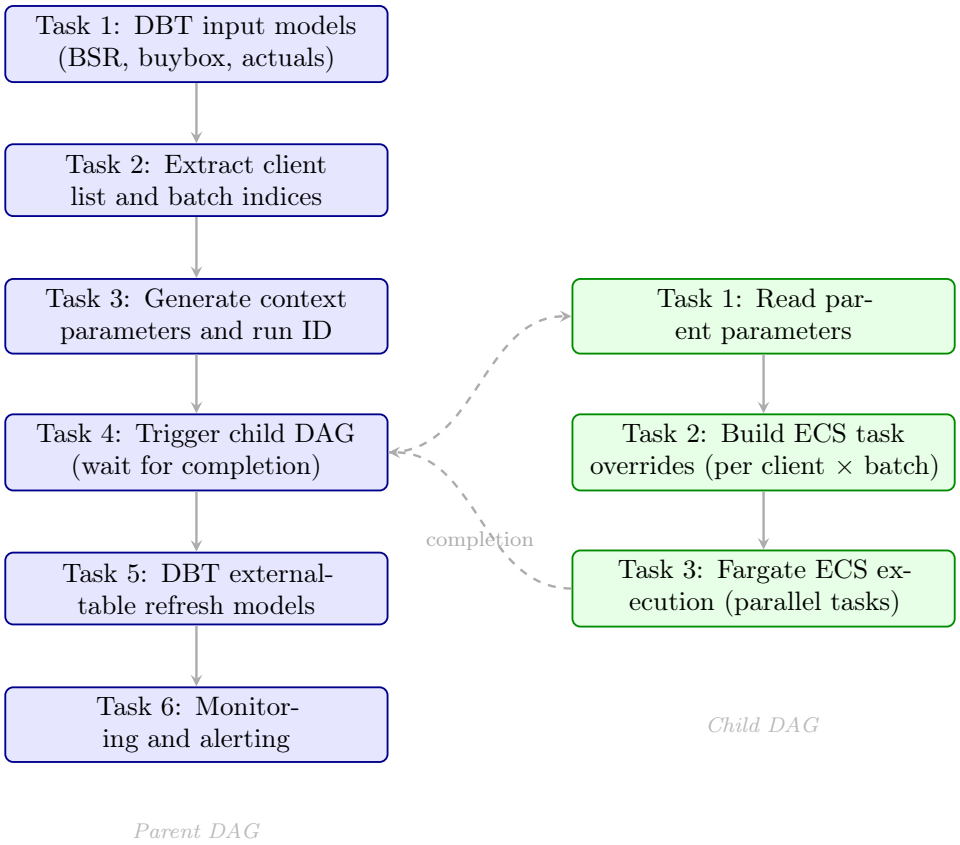

Figure 5.2: Two-level Airflow DAG structure. The parent DAG (*sh_ai_marketshare_pipeline_dag*) prepares data, discovers clients and batch indices, triggers the child DAG, and finalises outputs. The child DAG (*sh_ai_fargate_marketshare_service_dag*) builds per-client ECS overrides and launches parallel Fargate tasks. The parent waits for the child to complete before proceeding to post-processing.

5.3.1 Parent DAG

The parent DAG (*sh_ai_marketshare_pipeline_dag*) is the weekly entry point. It receives a reference date as parameter and executes six sequential tasks.

Task 1 — DBT input models. Executes the DBT transformation models that prepare the three input tables: the main BSR catalogue (with dimension attributes and batch indices embedded), the buybox table, and the actuals table. This step eliminates the need for a separate client configuration table: the set of seeds to process is inferred directly from the content of the BSR table for the given date, rather than read from a static configuration spreadsheet as in System 0.

Task 2 — Client and batch-index extraction. Runs a SQL query against the output of Task 1 to extract the distinct list of client identifiers and batch indices present for that

date. This list drives the downstream parallelisation: one Fargate task will be launched per client-and-batch-index combination.

Task 3 — Context parameters. Reads the input date from the DAG parameters and generates a unique run identifier for the entire weekly execution. This identifier propagates through all downstream tasks and into every output record, enabling full traceability from a dashboard row back to the Airflow run that produced it.

Task 4 — Trigger child DAG (blocking). Passes the client list, batch-index list, reference date, and run identifier to the child DAG, then waits for the child DAG to complete before proceeding. This is the only parent task with an external dependency.

Task 5 — DBT external-table refresh. Once all estimates have been written to cloud storage by the child DAG, this task executes the DBT models that read those outputs and refresh the downstream warehouse tables consumed by the dashboards.

Task 6 — Monitoring and alerting. Triggers the configured alerting workflow to notify the team of anomalies in the execution or in the output metrics.

5.3.2 Child DAG

The child DAG (*sh_ai_fargate_marketshare_service_dag*) performs the actual market share computation. It receives the parameters passed by Task 4 of the parent and executes three tasks.

Task 1 — Read parent parameters. Parses the configuration passed from the parent: client list, batch-index list, reference date, and run identifier.

Task 2 — Build ECS task overrides. Iterates over every client-and-batch- index combination and constructs an ECS task override for each, specifying the command-line arguments the Fargate container will receive: client identifier, batch index, reference date, and run identifier.

Task 3 — Fargate ECS execution. Launches all overrides in parallel using Airflow's dynamic task mapping. Each parallel task spins up a dedicated Fargate container that executes *Pipeline Complete* with its assigned parameters. Containers run independently with no shared state; parallelism is bounded only by the ECS cluster capacity.

This two-level structure cleanly separates concerns: the parent DAG owns the data-preparation and post-processing lifecycle, while the child DAG owns the computational execution. Adding a new client or batch index requires no DAG code change — it is handled automatically by Task 2 of the parent and the dynamic task mapping in Task 3 of the child.

5.4 Code Refactoring: Eliminating the Per-Seed Query Bottleneck

The Airflow DAG described in Section 5.3 defines *what* runs and *when*; this section describes how the code inside each Fargate task was redesigned to make it viable at scale. The refactoring was a response to a concrete bottleneck: at the seed volumes required by RMM, the original data retrieval design made weekly production delivery infeasible. The solution emerged in two successive decisions, each addressing a different layer of the problem, with an operational coda that resolved the memory constraint introduced by the second.

5.4.1 The Starting Point: Data Retrieval Distributed Across Every Pipeline Run

In the original system, each invocation of *Pipeline* was self-sufficient in terms of data access: it opened its own database connection and issued all the reads it needed before performing any computation. This was architecturally clean in isolation — each run was independent — but it produced a severe efficiency problem at scale.

A single *Pipeline* invocation issued a minimum of ten database reads and up to twenty-two in adverse conditions (≈13 in typical conditions). The sequence was: a store-name lookup to determine the retailer; a seed-count query for logging; a BSR query that, if the target date returned no data, triggered a retry loop of up to twelve additional attempts across a ±6-day tolerance window; a full-table actuals pull filtered downstream in Python; a buybox query; a full-table scan of the product-variant mapping; and, at the other end of the pipeline, four separate dimension lookups in the results step to retrieve store country, seed category, brand, and client name.

Several of these queries were structurally expensive beyond their count. The actuals pull retrieved all records for an entire retailer and filtered in Python, transferring far more data than necessary. The product-variant table was scanned in full for every seed. For Amazon seeds, an additional fan-out multiplied costs further: the original orchestrator launched three separate pipeline variants per Amazon combination — one per modelling approach — each with its own complete read stack.

With 20,000 seed-and-location combinations, the total number of Snowflake queries per weekly execution exceeded 500,000. The cumulative wait time in the shared database queue — not the computation itself — dominated total runtime. The system was not slow because the models were slow; it was slow because the data retrieval was structurally misaligned with the scale it was asked to serve.

5.4.2 First Decision: Consolidating Reads Inside the Pipeline

The first refactoring did not remove Snowflake access from *Pipeline*; it collapsed the many small reads into a minimal fixed set applied selectively by retailer type.

The most impactful change was the elimination of the BSR retry loop. The up-to-thirteen sequential queries were replaced by a single range query that retrieved BSR data for the target date and its entire tolerance window in one round-trip, with date-selection logic moved into Python after the data was received.

Equally impactful was the elimination of the four dimension lookups in the results step. In the original design, after the model had run and predictions were ready, the pipeline re-queried the database to retrieve store country, seed category, brand, and client name to build the output contract. In the redesigned system, these attributes were treated as columns already present on the BSR row, carried through the pipeline in the working dataframe. The four queries were removed entirely; the contract was built from in-memory data alone. This was possible because the upstream BSR table had been extended to include these dimension attributes directly, removing the need for runtime joins.

For non-Amazon seeds, the redesign reduced the per-invocation read count to **one query**:

the BSR range query. For Amazon seeds it remained at **three** — BSR, buybox, and actuals — now isolated in a dedicated Amazon import module called only when the seed was identified as Amazon. The full-table product-variant scan was also dropped, having been identified as an infrequently used input that did not justify a per-seed full-table read. The Amazon triple-pipeline fan-out was removed simultaneously: the orchestrator was redesigned to route each seed to a single pipeline determined by its signal availability.

At 20,000 seeds, these changes reduced the total weekly query count from more than 500,000 to approximately 40,000–60,000 reads. This was a substantial improvement, but instrumentation introduced at this stage — the first explicit per-step wall-clock measurement — revealed that the residual bottleneck was not computation but the overhead of opening database connections and queuing across tens of thousands of parallel invocations. Further progress required a more fundamental change.

5.4.3 Second Decision: Moving All Data Access Out of the Pipeline

The second refactoring was more radical. Rather than reducing queries per *Pipeline* invocation, it eliminated database access from *Pipeline* entirely, relocating all data retrieval to *Pipeline Complete* before any parallel work begins.

The mechanism is a single bulk SQL query, executed once per client batch, that retrieves the entire weekly estimation panel in one Snowflake round-trip. This query, structured as a chain of Common Table Expressions (CTEs), applies the sponsored-placement exclusion and best-execution date-tolerance logic previously handled by Python retry loops, joins actuals and buybox data conditionally for Amazon seeds, and pivots four weeks of historical signals into a wide-format panel. The result is a single dataframe containing every seed-and-location row the client needs for a given week, with all signal history and dimension attributes attached.

Pipeline Complete groups this dataframe by seed-and-location identifier, producing one slice per combination. Each slice is passed directly to a *Pipeline* worker as its sole data input. *Pipeline* no longer imports the database client module; its first step sorts the slice by rank and optionally persists a local diagnostic copy — no query is issued. All subsequent steps operate on the in-memory slice through seven stages: tier assignment, feature engineering, prediction, inference, optional evaluation, and results writing.

The observable effect on the step-file structure is significant. Before examining the step-by-step changes, Table 5.1 summarises the shift in what each component receives as input — the most fundamental architectural difference between the two systems.

Table 5.2 shows how the step files evolved from the original to the current system.

Because the pipeline numbering changed between the two systems, Table 5.3 gives an explicit mapping between the System 0 pipelines and their System 2 counterparts, to avoid ambiguity when the same number refers to different models across chapters.

Figure 5.3 illustrates the resulting execution model inside a single Fargate task, showing how *Pipeline Complete* and *Pipeline* relate to each other and to cloud storage.

The instrumentation was extended to cover the bulk import as a distinct phase with its own wall-clock measurement and peak RSS reporting. This revealed a shift in the bottleneck: the dominant cost was no longer the aggregate of many small queries but the single large dataframe

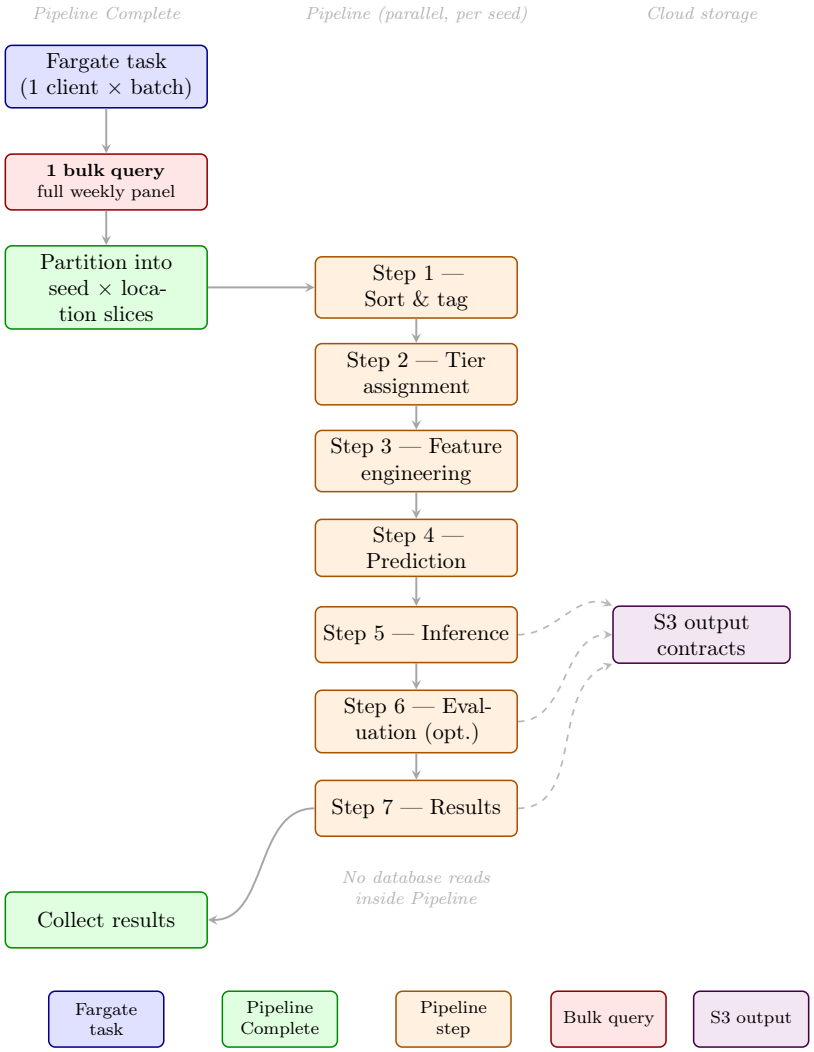

Figure 5.3: Execution model inside a single Fargate task. *Pipeline Complete* issues one bulk query, partitions the result into per-seed slices, and dispatches parallel *Pipeline* workers. Each worker receives a pre-loaded in-memory slice and performs no database reads. Steps 5–7 write output contracts to S3.

Table 5.1: Input comparison between System 0 and System 2 for the two code components. In System 0, Pipeline Complete queries the configuration table to discover seed-and-location combinations; Pipeline then queries Snowflake individually for each combination. In System 2, Pipeline Complete issues one bulk query that returns a complete dataframe; Pipeline receives a pre-partitioned in-memory slice with no database access.

Component	System 0 input	System 2 input
Pipeline Complete	Client ID + date → queries config table for all seed × location combinations	Client ID + date → issues 1 bulk query; receives full weekly panel as dataframe
Pipeline (per seed)	Client ID + seed ID + location ID + date → queries Snowflake for all data needed for this seed (≈13 queries)	In-memory dataframe slice (BSR + actuals + buybox + dims) → no database access

Table 5.2: Evolution of per-seed pipeline step files. The original system loaded all data inside Step 1 and issued four dimension queries in Step 6. The current system receives a pre-loaded dataframe slice and performs no database reads.

Original step	Current step	Change
Step 1 — BSR + actuals + buybox + product-variant (10–22 queries)	Step 1 — sort by rank, tag execution ID	All queries removed
Step 2 — feature engineering	Step 3 — feature engineering, per pipeline	Split by pipeline; moved to step 3
Step 3 — orchestrator (route to pipeline)	Step 2 — tier assignment (Tiers 1–9 → pipeline)	Renumbered; tier logic extended
Step 3 — prediction (pipelines 1–4)	Step 4 — prediction (pipelines 1–5)	Renumbered; pipeline 5 added
Step 4 — inference	Step 5 — inference + revenue proxy	Renumbered; proxy added
Step 5 — evaluation	Step 6 — evaluation (optional)	Renumbered
Step 6 — results + 4 dimension queries	Step 7 — results (no queries)	Dimension lookups eliminated
—	`query.sql` + bulk import module	New: one SQL for entire client panel

transfer from Snowflake into Python memory.

5.4.4 Operational Coda: Batch Indexing for Memory Safety

The second architectural decision eliminated per-seed queries but introduced a new constraint: loading the complete weekly panel for a large client in a single `pd.read_sql` call transfers the entire dataset into the Fargate container's memory at once. For clients with a large seed universe — most notably RMM, which tracks more than 20,000 seeds across four US retailers — the resulting dataframe exceeded the memory limit of the ECS Fargate task, causing out-of-memory failures.

The solution was to partition the client's seed universe into fixed-size batches and process each batch as a separate Fargate task. The partition is defined by a field called `index_seed`, computed at DBT build time and embedded directly in the `fact_plp_bsr` table alongside the BSR data. Each product row in the BSR table is assigned an integer batch index that groups seeds into batches of approximately 2,500. For RMM with more than 20,000 seeds, this yields roughly 8–10 batches per client per week.

Table 5.3: Mapping between System 0 pipelines and System 2 pipelines. The XGBoost model for rank-and-engagement seeds (Pipeline 3) was retrained as a new model in System 2; the rank-only XGBoost pipeline (Pipeline 4) is entirely new; and the equal-share assortment fallback, which is not a predictive AI model, was renumbered from Pipeline 4 to Pipeline 5.

System 0 pipeline	System 2 pipeline	Tier(s)
Pipeline 1 — actuals + NS (log-linear)	Pipeline 1 — rank, NS, actuals (calibrated)	1
Pipeline 2 — NS, no actuals	Pipeline 2 — rank, NS, no actuals	2
Pipeline 3 — rank + engagement (XGBoost)	Pipeline 3 — rank + engagement (new XGBoost)	5
— (not present)	Pipeline 4 — rank only (new XGBoost)	6
Pipeline 4 — equal-share fallback	Pipeline 5 — equal-share fallback	7–9

Both the bulk query and the Python code were adapted to handle this partitioning correctly. The bulk query accepts an optional `index_seed` filter that restricts the data load to a single batch's rows. *Pipeline Complete* receives the batch index as a command-line parameter passed by the Airflow child DAG and injects it into the query at runtime. The Airflow child DAG (Task 2) generates one ECS task override per client-and-batch-index combination, so the full seed universe is covered by multiple parallel Fargate tasks running concurrently, each loading a memory-safe subset.

This design makes a deliberate trade-off: the "one query per client" ideal becomes "one query per client-and-batch," but the total query count remains orders of magnitude lower than the per-seed architecture. After the bulk dataframe is loaded, it is explicitly deleted and the garbage collector is invoked before the parallel thread dispatch begins, preventing the full panel from occupying memory during execution. The peak RSS observed in a production run with batch indexing for RMM was approximately 14 GB, well within the capacity of the Fargate task definition, with the bulk import phase accounting for the dominant share.

The batch-index mechanism is treated as interim operational plumbing rather than a permanent architectural feature. The designed long-term resolution involves server-side export of the bulk result directly to cloud storage, bypassing the in-memory transfer step entirely; this is left as future work (Section 9.5).

5.4.5 Conditional Loading of Amazon-Specific Signals

A further design principle embedded in the bulk query is the conditional treatment of Amazon-specific signals. The query identifies Amazon seeds via a string match on the store name field and attaches an Amazon flag to every row. Actuals and buybox data are joined using an inner filter on that flag, followed by an outer join onto the full catalogue. Non-Amazon rows receive null values in those columns with no Python-side branching required before tier assignment: the tier system (Chapter 6) interprets null actuals and buybox columns as absent signals and routes accordingly.

At the results stage, for Amazon seeds where all actual columns are non-null, the pipeline replaces model predictions with ground-truth actuals. The manufacturing-view actuals (1P + 3P combined, integrated during this project) take priority where available, followed by the

sourcing-view actuals (1P only), with model predictions used as the fallback for products or weeks not covered by Vendor Central data.

5.5 Designed Migration to Apache Iceberg

5.5.1 Context: Limitations of the Current Output Storage Approach

In the current production system, the output contracts produced by Steps 5, 6, and 7 of the Pipeline are written as Parquet files to cloud storage, partitioned by run identifier, client, seed, and location. These files are consumed by Snowflake through *External Tables* — a mechanism that points a Snowflake table definition at a storage path and makes the files queryable through standard SQL.

While functional, this approach accumulated four operational limitations that became increasingly visible as the system scaled.

Manual refresh requirement. External Tables do not automatically reflect new files written to storage. Every time the pipeline writes new Parquet outputs, a refresh must be triggered to make the new data visible in Snowflake. In the current setup this is handled by a pre-hook in the downstream DBT models, but it introduces a dependency and a potential failure point that is absent from a native table format.

Dependency on the Data Engineering team for schema changes. Modifying the definition of an External Table — adding or removing columns, changing types, updating the storage path — requires changes in a shared infrastructure repository managed by the Data Engineering team. Any schema evolution in the AI pipeline output triggers coordination overhead that delays delivery.

Manual versioning through folder structure. The current system manages output versions by maintaining separate storage path prefixes per step version. Each version increment requires creating a new folder structure, updating the External Table definition to point at it, and maintaining old versions for downstream compatibility. This process is error-prone, not atomic, and carries no built-in audit trail.

No fault tolerance on schema mismatch. If a pipeline execution writes a file with a column set that differs from what the External Table expects — due to a code change, a missing field, or a type mismatch — the table breaks at query time rather than at write time, with no enforcement at ingestion.

5.5.2 Apache Iceberg as the Target Format

Apache Iceberg is an open table format designed for large-scale analytic datasets stored in object storage [13]. Unlike External Tables, which are a thin SQL view over a folder of files, an Iceberg table maintains an internal metadata layer — a tree of manifest files and snapshot records — that tracks every write operation atomically. This metadata layer is what enables the properties that make Iceberg attractive for this use case.

Automatic refresh. Every write to an Iceberg table updates the metadata layer atomically; Snowflake can reflect new data within approximately 30 seconds with no manual refresh step required.

Schema evolution without Data team coordination. Iceberg supports adding and dropping columns directly from a dedicated schema management repository owned by the AI team, without requiring changes to shared infrastructure, subject to a lightweight review process with the Data Engineering team to ensure downstream compatibility.

Snapshot-based version control. Every write creates an immutable snapshot. Historical data can be queried at any previous snapshot, enabling time-travel queries, rollback of erroneous executions, and transparent audit — replacing the manual folder-based versioning entirely.

Schema enforcement at write time. Iceberg enforces the declared schema on every write, rejecting mismatched data before it reaches Snowflake.

Figure 5.4 illustrates the technical architecture of the integration. Data written by the Python pipeline in Parquet format lands in cloud storage and is registered in the AWS Glue Catalog, which acts as the Iceberg metastore. Snowflake reads the table through the Glue Catalog via an External Table connector, making the Iceberg table queryable through standard SQL with automatic refresh. The AI team manages table definitions — creation, schema evolution, and IAM role configuration for write permissions — through a dedicated repository (*snowflake-ai-services*), keeping this responsibility separate from the Python estimation code.

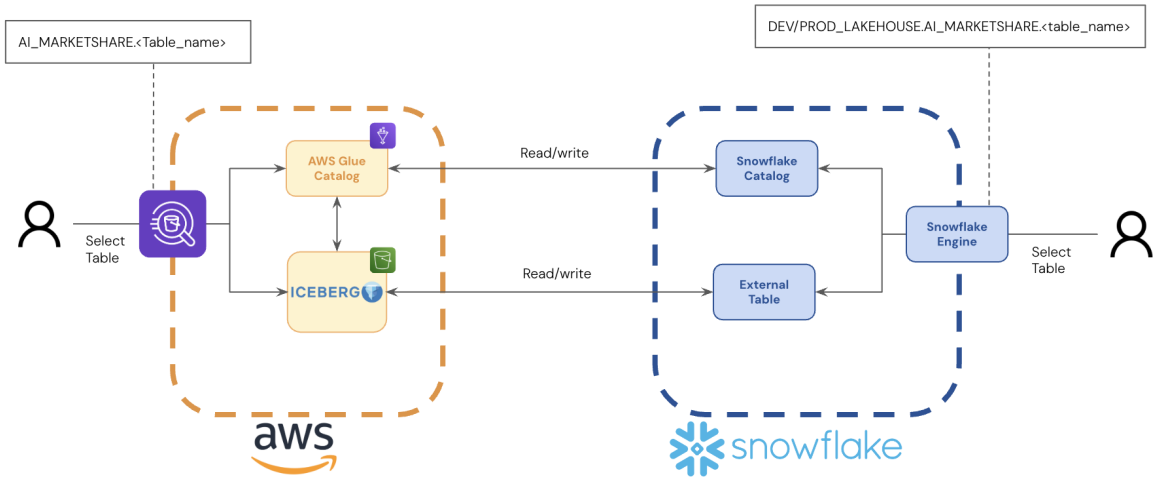

Figure 5.4: Apache Iceberg integration architecture. On the AWS side (orange dashed boundary), the AI team writes data via PyIceberg; the AWS Glue Catalog acts as the Iceberg metastore, tracking every write as an atomic snapshot. On the Snowflake side (blue dashed boundary), an External Table connector reads the Glue Catalog metadata and exposes the Iceberg table through the standard Snowflake Engine, making it queryable with SQL without any manual refresh step. Tables are accessible under the namespaces `AI_MARKETSHARE` (AWS side) and `DEV/PROD_LAKEHOUSE.AI_MARKETSHARE` (Snowflake side).

Figure 5.5 shows how this Iceberg layer integrates into the full system data flow, replacing the Parquet + External Tables path of the current production system.

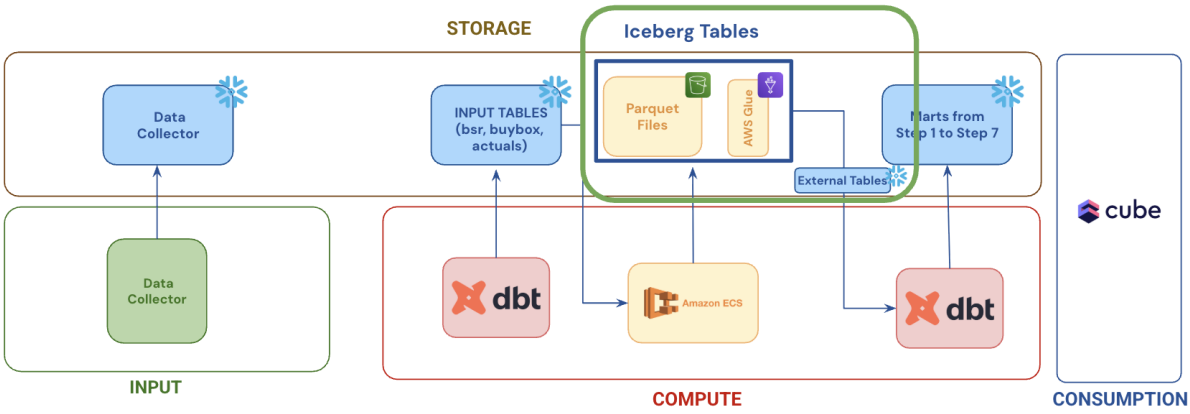

Figure 5.5: System 2 data flow with the designed Iceberg migration applied. The Parquet Files and External Tables artefacts of the current system (cf. Fig. 5.1) are replaced by a single Iceberg Tables layer (green border) containing Parquet files managed by the AWS Glue Catalog. The Iceberg layer is read by Snowflake via the existing External Tables connector with automatic refresh, eliminating the manual refresh requirement. All other domains (Input, Compute, Consumption) remain unchanged.

5.5.3 Implementation Status and the Concurrency Blocker

The Iceberg migration has been fully designed and implemented on a dedicated development branch of the estimation repository. The three pipeline steps that write outputs (Steps 5, 6, and 7) have been adapted to use a new ingestion function that authenticates via an IAM role and uses the PyIceberg library to write data directly to the Iceberg table, bypassing the previous Parquet-to-storage path. The development environment has been extended accordingly: the Iceberg tables exist in the development schema, can be queried directly from Snowflake, and the pipeline runs end-to-end against them. The secrets configuration has also been restructured, separating credentials from environment-specific configuration variables to support clean switching between development and production.

Despite this, the branch has not been merged into production. The blocker is a fundamental conflict between the Iceberg write model and the parallel execution architecture of the system.

Iceberg uses *optimistic concurrency control* to ensure atomic commits. When a writer commits a new snapshot, it reads the current table metadata, appends its data files, and attempts to write a new metadata pointer. If another writer has committed between the read and the write — which is the expected case when multiple Fargate tasks execute concurrently — the commit fails. In the current architecture, multiple Fargate tasks run in parallel for different client-and-batch-index combinations, and within each task, multiple threads execute Pipeline workers simultaneously. All of these threads and tasks attempt to write to the same Iceberg tables at the same time, producing commit conflicts that make ingestion unreliable under load. This is the precise tension between the two goals that were pursued at the same time: maximum parallelism for scale, and transactional consistency at the storage layer.

Table 5.4 summarises the conflict.

5.5.4 Candidate Resolutions

Three resolution strategies have been identified, each addressing the conflict at a different layer.

Partitioned writes: one Iceberg table per client or batch. If each Fargate task writes

Table 5.4: Conflict between the parallel execution model and the Iceberg write model.

	Parallel execution	Iceberg write model
Goal	Maximise throughput via concurrent seed estimations	Guarantee atomicity via snapshot commits
Mechanism	Multiple Fargate tasks and threads writing simultaneously	Optimistic concurrency: reads metadata, then attempts atomic commit
Conflict	Multiple writers target the same table at the same time	Commit fails when metadata has been updated by another writer

to a dedicated Iceberg table, concurrent commits would not conflict because each writer targets a different table object. Downstream models would need to aggregate across tables, but the commit semantics would be safe. This is the most straightforward approach given the current architecture.

Serialised commit queue. A lightweight coordinator service could accept payloads from all parallel workers and commit them to the shared Iceberg table sequentially, decoupling parallel computation from the serial commit requirement. This preserves the shared-table model but introduces an additional service.

Server-side export with deferred registration. Each parallel task exports its output to a separate storage prefix, as today, but uses the Iceberg metadata API to register those files in the table's manifest as a single atomic batch operation after all parallel tasks complete. This mirrors the current flow most closely and avoids the parallel-commit problem by deferring the metadata update to the sequential post-processing phase.

Resolution of this blocker and the production deployment of the Iceberg migration are identified as priority future work (Section 9.5).

Chapter 6

Modeling: Tier-Based Orchestration and Pipelines

This chapter describes the modeling layer of the market share estimation service: how each seed is routed to the most appropriate estimation strategy, how each of the five pipelines works, and how the revenue proxy enables aggregation across the category hierarchy. Together with Chapter 5, this constitutes the core technical contribution of the thesis.

6.1 Problem Formulation

Market share in the digital shelf context is the proportion of total category sales captured by a given product or brand within a single Product Listing Page (PLP). For each seed (a PLP-and-location combination) in a given week, the system must estimate the sales volume of every organic product in the ranking and derive each product's share of the total.

The estimation problem admits two distinct regimes depending on the signals available. When strong demand proxies exist — in particular the NumberSells (NS) indicator or Vendor Central actuals — the system estimates *absolute units* sold per product and derives share by normalisation. When only rank and engagement signals are available, absolute sales cannot be reliably inferred and the system estimates *relative market share* directly as a percentage that sums to one within the PLP universe. The two regimes are handled by different pipelines, selected automatically by the tier system described in the next section.

A key observation that justifies both regimes is that the rank-to-sales relationship within any given PLP follows a power-law decay: the top-ranked product captures a disproportionately large share, and the curve is relatively stable across categories and weeks [4]. This means that even when absolute volumes are unknown, the *shape* of the distribution is learnable from rank signals alone, which is the basis for the XGBoost pipelines.

6.2 The Tier System and Automatic Routing

The central design decision in the modeling layer is that a single universal model is not adequate across the heterogeneous signal environments described in Section 3.1.1. The orchestrator —

Step 2 of the Pipeline — inspects the signals available in the slice for each seed and assigns it to one of nine tiers, each mapped to a specific pipeline.

Table 6.1 shows the tier definitions and their pipeline assignments. The assignment logic evaluates signals in priority order: Amazon seeds with the richest signal environment (rank, NS, and actuals) are assigned to Tier 1; each successive tier relaxes one or more signal requirements. Tiers 3 and 4 identify Amazon seeds that have rank but lack NS — a coverage gap where no pipeline is currently assigned.

Table 6.1: Tier assignment by available signals and the pipeline each tier is routed to. A dash (—) in a signal column means the signal is not required for that tier. Tiers 3 and 4 have no assigned pipeline and are a known coverage gap. f^* refers to the decision score defined in Eq. (6.1).

Tier	Amazon	Rank	Reviews / Ratings	NS	Actuals	Pipeline
1	yes	yes	—	yes	yes	1
2	yes	yes	—	yes	no	2
3	yes	yes	—	no	yes	—
4	yes	yes	—	no	no	—
5	no	yes	yes ($f^* > 0$)	—	—	3
6	no	yes	no ($f^* \leq 0$)	—	—	4
7	no	no	yes	yes	—	5
8	no	no	yes	—	—	5
9	no	no	no	—	—	5

The boundary between Tiers 5 and 6 — and thus between Pipelines 3 and 4 — is determined by a linear score over the null rates of review and rating signals:

$$\text{score} = -0.3690 \cdot p_{\text{null,reviews}} - 1.5230 \cdot p_{\text{null,ratings}} + 51.2840 \tag{6.1}$$

where $p_{\text{null,reviews}}$ and $p_{\text{null,ratings}}$ are the percentage of null values for customer reviews and ratings within the seed's execution, expressed as percentages (0–100). Seeds with score > 0 are routed to Pipeline 3 (which uses engagement features); seeds with score ≤ 0 are routed to Pipeline 4 (rank only). This decision function was derived empirically from the threshold analysis described in Section 6.3.4 and Section 6.3.5.

6.3 Estimation Pipelines

6.3.1 Pipeline 1 — Tier 1: Rank, NumberSells, and Actuals

Pipeline 1 handles Amazon seeds for which all three signal types are available: BSR rank, NumberSells, and Vendor Central actuals. It extends Pipeline 2's isotonic rank-to-units curve with a calibration layer that anchors predictions to real sales.

The calibration operates by fitting a rank-calibration ratio from the NS-based prediction and the observed actuals across both the current week and historical weeks. A decreasing isotonic regression maps rank to a ratio scale, yielding a multiplicative adjustment factor:

$$\hat{u}_i = \widehat{NS}_{\text{weekly},i} \times \text{RANK\_CALIBRATION\_RATIO}(rank_i) \tag{6.2}$$

For products whose manufacturing actuals are fully present (1P + 3P), those actuals are used

directly as the prediction rather than the model output, ensuring that ground-truth data takes precedence whenever available. The minimum number of actuals required to fit the calibration is three observations; below this threshold the pipeline falls back to Pipeline 2 behaviour.

Pipeline 1 serves as the ground-truth reference for the system: its outputs are used as pseudo-labels when training Pipelines 3 and 4 offline.

6.3.2 Pipeline 2 — Tier 2: Rank and NumberSells, No Actuals

Pipeline 2 covers Amazon seeds where NS is available but Vendor Central actuals are not. It is the foundational estimation model for Amazon seeds without ground-truth calibration.

An important limitation of the System 0 architecture was that Pipeline 2 (and Pipeline 1) were only executed for products that displayed the NumberSells indicator on the PLP — products without NS were simply skipped. In practice, a PLP can contain a mix of products with and without NS, and the absence of the indicator for a product does not imply zero sales; it simply means the retailer is not displaying the counter for that product at that time. Limiting estimation to NS-bearing products therefore produced incomplete PLP coverage and underrepresented some competitors within the same category. The new architecture addresses this by using the tier system to route NS-bearing products to Pipelines 1 and 2 and routing the remaining products — those without NS but with rank — to the XGBoost pipelines (Tiers 5 or 6), ensuring that every organic product in the PLP receives a market share estimate.

The core model of Pipeline 2 is a *decreasing isotonic regression* mapping rank to weekly NS (absolute units), fitted on the current week augmented with four weeks of historical data. This five-point time series per product makes the estimated rank-to-sales curve more stable against weekly noise.

The model supports several curve forms (isotonic, log-linear, power-law, mid-point), with isotonic regression as the default. Pseudo-targets are derived from NS midpoint values, with a small epsilon anchor at the maximum rank to enforce the monotone constraint.

Predictions are adjusted post-hoc by the Inertia Score — the momentum signal inherited from System 0 (Section 4.4.2) — clipped to a [0.5, 2.0] multiplicative range to prevent extreme adjustments from promotional spikes or transient rank movements.

6.3.3 Pipeline 3 — Tier 5: XGBoost with Rank and Engagement Features

Pipeline 3 is one of the two new models developed in this thesis. It targets non-Amazon seeds for which both rank and engagement signals (reviews and ratings) are sufficiently available (Tier 5). Rather than estimating absolute units, it estimates relative market share as a log-transformed percentage.

Training. The model is an `XGBRegressor` trained offline on historical execution data. The training target is $\log(MS_{\text{real}})$, derived from Pipeline 1 outputs acting as pseudo-labels. Features include rank-relative measures, standardised review and rating signals within each execution, distribution-level statistics aggregated across the execution (descriptor and brand-descriptor features), and review trend slopes. The training procedure uses:

- Hyperparameter optimisation with Optuna over 50 trials;
- Group-aware cross-validation with GroupKFold ($k = 5$, grouped by execution) to prevent data leakage across PLPs;
- Sample weights of 2/rank for products with observed actuals, upweighting data-rich observations.

Duan's smearing correction. Because the target is log-transformed, naive back-transformation by exponentiation introduces a systematic underestimation bias (Jensen's inequality). Duan's smearing estimator [17] corrects for this:

$$\widehat{MS}_i = \exp(\hat{y}_i) \times \frac{1}{n} \sum_{j=1}^{n} \exp(\hat{\varepsilon}_j) \tag{6.3}$$

where $\hat{\varepsilon}_j$ are the out-of-fold residuals from cross-validation. The smearing factor is stored in the model metadata and applied at inference time. After back-transformation, predictions are normalised to sum to one within each execution.

New vs. legacy model. Evaluations conducted on held-out production executions compare the legacy Pipeline 3 model with the new version developed in this project. Table 6.2 reports the results on both the in-sample training set and a temporally disjoint held-out dataset.

Table 6.2: Pipeline 3: legacy vs. new model performance. In-sample results on the training set (1,483,843 rows) and held-out results on a temporally disjoint execution set. Metrics are at SKU-and-execution level; brand-level metrics are in parentheses. WMAPE is the primary metric; lower is better. R^2 higher is better.

	In-sample		Held-out	
Model	WMAPE (%)	R^2	WMAPE (%)	R^2
Legacy (Pipeline 3)	98.0 (66.5)	0.23 (0.55)	101.4 (69.5)	0.23 (0.52)
New (Pipeline 3)	32.1 (16.7)	0.92 (0.97)	45.9 (26.2)	0.73 (0.92)

The new model reduces WMAPE by approximately 67% in-sample and 55% on held-out data at SKU level, with even larger improvements at brand level (75% and 62% respectively). The R^2 improvement from 0.23 to 0.92 in-sample confirms that the new feature set and training procedure capture substantially more of the variance in market share. Figure 6.1 illustrates this improvement on a representative execution: the legacy model assigns nearly identical flat market shares to all products regardless of rank, while the new model correctly recovers the steep power-law decay observed in ground-truth data.

Figure 6.1: In-sample spot-check for a representative execution. Left: all 2,000+ ranked products. Right: top 50 products. The legacy Pipeline 3 model (orange squares) predicts nearly uniform market shares across all ranks. The new model (green triangles) correctly captures the power-law decay and closely tracks the ground-truth values (blue circles), particularly in the commercially critical top-ranked positions.

6.3.4 Pipeline 4 — Tier 6: XGBoost with Rank Only

Pipeline 4 is the second new model. It targets non-Amazon seeds where engagement signals are too sparse for Pipeline 3 to be reliable (Tier 6). Its feature set contains only rank-derived variables: absolute rank, log-transformed rank, and relative rank within the execution. The same Optuna + GroupKFold + Duan's smearing training recipe is used, applied to a rank-only feature matrix.

Threshold analysis. The boundary between Tiers 5 and 6 was derived through a systematic null-rate experiment. A set of high-quality executions was selected and synthetic null rates for review and rating features were injected at a grid of values ranging from 0% to 100% in 5-percentage-point steps. At each grid cell, WMAPE was computed for both Pipeline 3 and Pipeline 4. A logistic regression decision boundary was then fitted on the binary outcome (Pipeline 3 better vs. Pipeline 4 better) using the two-dimensional null-rate space as input.

The resulting decision function is Equation (6.1). When applied to the 448 real-world held-out executions, the decision boundary recommends Pipeline 3 for 446 of them, confirming that Pipeline 4 is the correct fallback only for genuinely data-poor seeds with very high null rates.

Performance comparison. Table 6.3 reports the four-model comparison on rank-only held-out data, where all models are evaluated with only rank features available to ensure a fair comparison across tiers.

Table 6.3: Four-model comparison on rank-only held-out data (temporally disjoint execution set). SKU-level and brand-level metrics (in parentheses). The new Pipeline 4 model substantially outperforms all baselines, including the new Pipeline 3 model in rank-only conditions.

Model	WMAPE (%)	R^2
Legacy Pipeline 3	115.0 (79.6)	0.04 (0.38)
New Pipeline 3	106.1 (66.6)	0.48 (0.66)
New Pipeline 4	63.0 (37.5)	0.48 (0.83)
Baseline ($1/N$)	156.7 (107.3)	0.04 (0.09)

Pipeline 4 achieves a 45% WMAPE reduction over the legacy Pipeline 3 at SKU level and 53% at brand level in rank-only conditions, confirming that a model trained specifically for sparse-signal environments is substantially more accurate than applying a richer model with missing features.

Figure 6.2 shows a visual comparison of all four models on a representative rank-only execution. The legacy Pipeline 3 and the naive baseline both produce nearly flat predictions. The new Pipeline 3 model improves the shape slightly, but still underestimates the steepness of the distribution. The new Pipeline 4 model best recovers the power-law decay from rank alone.

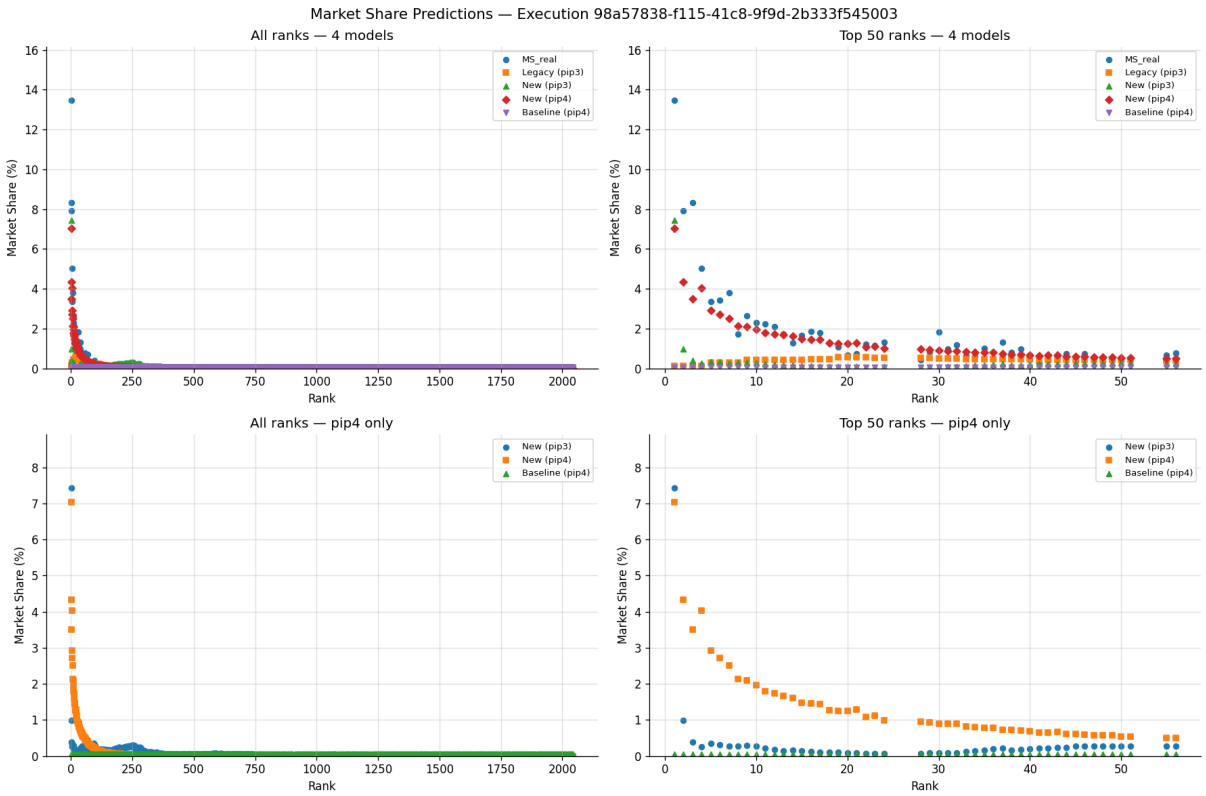

Figure 6.2: Visual comparison of four models on a rank-only held-out execution. Top row: all models including the legacy Pipeline 3 (orange squares) and the naive baseline (purple triangles). Bottom row: focus on the Pipeline 4 models. The new Pipeline 4 (orange squares, bottom) captures the power-law decay far better than the new Pipeline 3 (blue circles, bottom) or the naive baseline (green triangles, bottom) when only rank features are available.

Feature ablation. Table 6.4 reports Pipeline 3 performance under four conditions: full features, null ratings, null reviews, and rank-only. This ablation study quantifies the marginal contribution of each signal type and validates the routing decision. When ratings are absent, Pipeline 4 outperforms the null-ratings variant of Pipeline 3, justifying the tier split.

6.3.5 Null-Rate Threshold Study: Routing Between Pipelines 3 and 4

A key design question for Pipelines 3 and 4 is the routing boundary: given a seed with a certain proportion of null values in its review and rating columns, which model should be used? The answer is not obvious *a priori* — Pipeline 3 uses engagement features and should outperform Pipeline 4 when those features are available, but as null rates increase, the engagement features

Table 6.4: Pipeline 3 feature ablation on the held-out dataset. Performance degrades gracefully as signals are removed. Pipeline 4 is included for comparison. Brand-level WMAPE in parentheses.

Condition	WMAPE (%)	R^2
Pipeline 3, full features	45.9 (26.2)	0.73 (0.92)
Pipeline 3, null reviews	57.2 (33.6)	0.64 (0.88)
Pipeline 3, null ratings	82.3 (49.7)	0.19 (0.75)
Pipeline 3, rank only	106.1 (66.6)	0.48 (0.66)
Pipeline 4 (rank only)	63.0 (37.5)	0.48 (0.83)

become noise rather than signal, and Pipeline 4's rank-only approach may be preferable.

To answer this question empirically, a systematic null-rate injection experiment was conducted on both models.

Null-rate injection experiment. To derive a principled routing boundary, 100 high-quality held-out executions were selected (executions with minimal real null rates, ensuring that injected nulls represent controlled degradation rather than real data quality). A two-dimensional grid of synthetic null rates was then constructed: for each combination of % null reviews $\in \{0, 5, 10, \ldots, 100\}$ and % null ratings $\in \{0, 5, 10, \ldots, 100\}$ (441 grid cells in total), both models were evaluated on the modified executions and average WMAPE was recorded.

Figure 6.3 shows the resulting WMAPE heatmaps for each model across the null-rate grid. Pipeline 3 (left) is highly sensitive to null ratings: WMAPE rises steeply as ratings become sparse, reaching values above 100% when null rates exceed 60–70%. Pipeline 4 (right) is completely insensitive to null rates by design, maintaining a constant WMAPE across the entire grid.

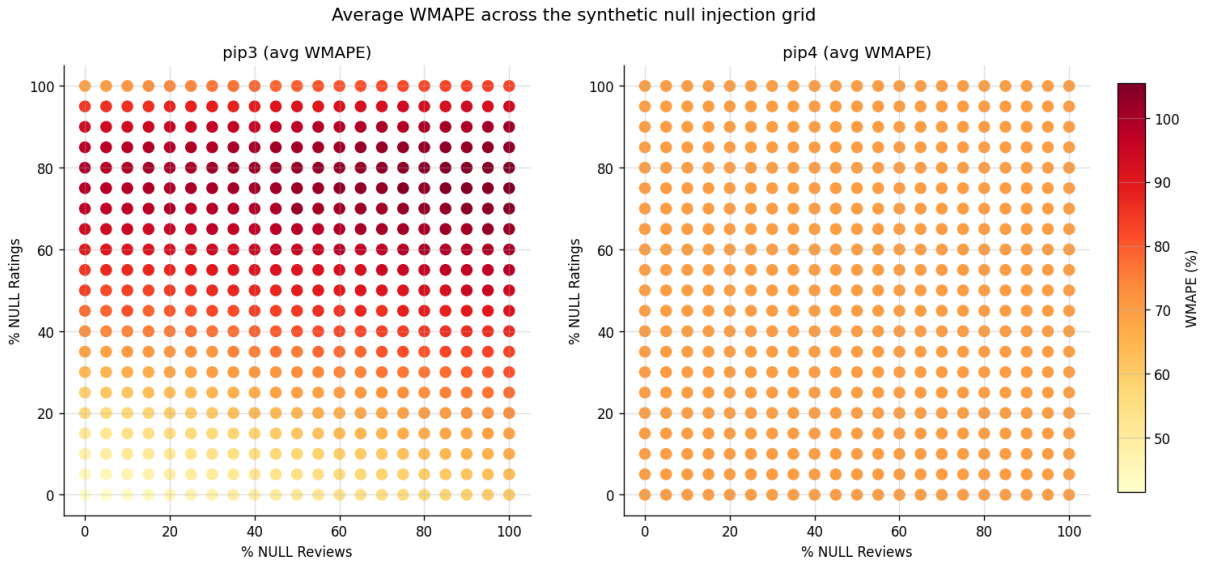

Figure 6.3: Average WMAPE across the 441-cell synthetic null-rate injection grid. Left: Pipeline 3 (engagement features). Right: Pipeline 4 (rank only). Pipeline 3 degrades sharply as null ratings increase, while Pipeline 4 is invariant to null rates. The crossover region — where Pipeline 4 becomes preferable — can be read from the left heatmap as the region where its colour exceeds the constant level of the right heatmap.

Decision boundary estimation. A logistic regression classifier was fitted on the binary outcome at each grid cell (Pipeline 3 better vs. Pipeline 4 better) using the two null-rate values as inputs. Two decision rules were derived: exact parity (Pipeline 3 strictly better) and a 5%-margin rule (Pipeline 3 at least 5% better in WMAPE), which is more robust to noisy grid cells.

The 5%-margin decision function is:

$$f(p_r, p_g) = -0.3690\, p_r - 1.5230\, p_g + 51.2840 \tag{6.4}$$

where p_r is the percentage of null reviews and p_g is the percentage of null ratings (both in $[0, 100]$). Pipeline 3 is preferred when $f > 0$; Pipeline 4 when $f \leq 0$. This is the function that implements Equation (6.1) in the orchestrator.

Figure 6.4 shows the decision boundary overlaid on the grid. The boundary is nearly horizontal in the ratings dimension, confirming that ratings null rate is the dominant factor in routing: even with very high review null rates, Pipeline 3 remains preferable as long as ratings are below approximately 35–40% null. Pipeline 4 becomes necessary only when ratings data is substantially absent.

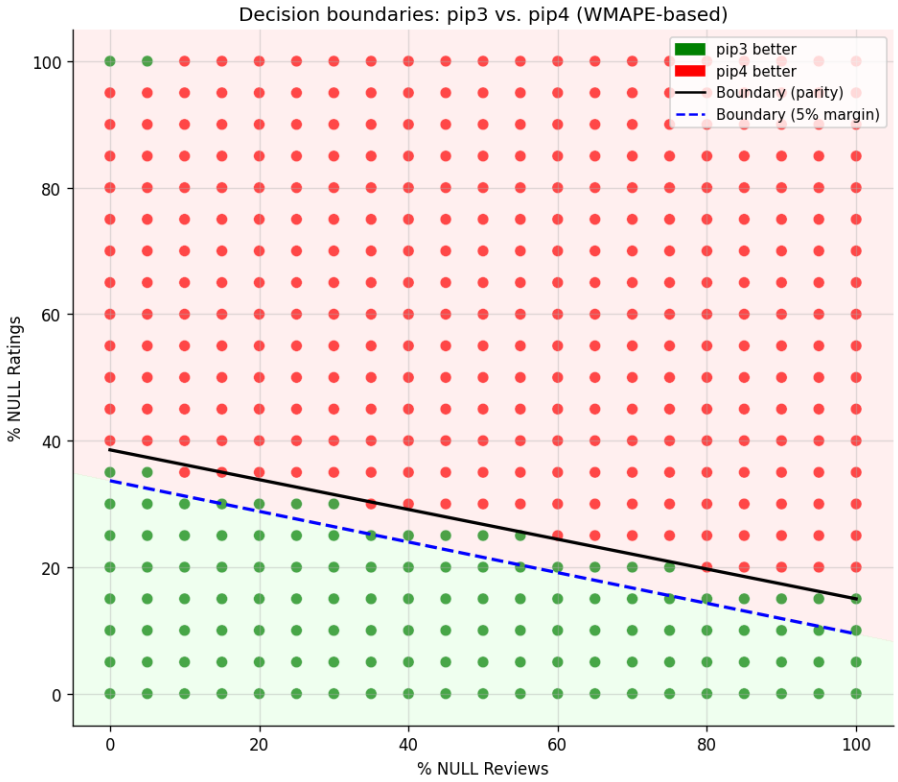

Figure 6.4: Decision boundary between Pipeline 3 (green region, lower-left) and Pipeline 4 (red region, upper-right) in the two-dimensional null-rate space. Solid black line: exact-parity boundary (Pipeline 3 strictly better). Dashed blue line: 5%-margin boundary (Pipeline 3 at least 5% better in WMAPE), used in production. The boundary is steeper in the ratings dimension, reflecting ratings' greater influence on Pipeline 3 performance.

When applied to the 448 real-world held-out executions, the 5%-margin decision boundary recommends Pipeline 3 for 446 of them, confirming that Pipeline 4 is the correct routing only for genuinely data-poor seeds.

6.3.6 Pipeline 5 — Tiers 7–9: Equal-Share Fallback

Pipeline 5 covers seeds where no rank signal is available, regardless of whether NS or engagement signals are present. It assigns equal market share to every product in the execution: $\widehat{MS}_i = 1/N$, where N is the number of products. This deterministic fallback ensures that the system always produces an output, at the cost of accuracy. Its purpose is continuity of service rather than precision, and it is not subject to quantitative evaluation in this thesis.

6.3.7 Tiers 3 and 4: Coverage Gap

Tiers 3 and 4 identify Amazon seeds that have rank but lack NumberSells, regardless of actuals availability. No pipeline is currently assigned to these tiers: when the orchestrator assigns a seed to Tier 3 or 4, execution stops after Step 2 and no prediction is produced.

No seeds of this type have been identified in the current production universe. Amazon US, the primary Amazon market covered by RMM, consistently displays the NumberSells indicator for the vast majority of ranked products, making Tiers 3 and 4 effectively empty in the current client base. However, Amazon in other countries — such as Amazon ES, Amazon DE, or Amazon IT — does not always expose NumberSells on PLPs. If in the future clients request market share estimation for seeds in these markets, Tiers 3 and 4 would become active, and a new pipeline would need to be designed to handle Amazon seeds with rank but without any NS signal. The most natural approach would be to calibrate directly against manufacturing actuals (1P + 3P) without NS, using the Vendor Central data that is already ingested for clients who have granted access. This extension is identified as a priority future improvement (Section 9.5).

6.4 Revenue Proxy and Market Share Aggregation

6.4.1 Motivation: From Subcategory to Category Market Share

The estimation pipelines described above produce market share at the level of the *subcategory* — the most granular Product Listing Page (PLP) tracked by the system, which is the scraping unit of RMM. For example, the seed *Grocery & Gourmet Food > Beverages > Bottled Beverages, Water & Drink Mixes > Soft Drinks* is a single subcategory seed. Within it, every product's market share sums to exactly 1 (100%).

Client reporting, however, frequently requires market share at a higher category level — for example, at *Beverages* or *Bottled Beverages, Water & Drink Mixes*. The aggregation problem is not equally complex for all pipelines. For seeds served by Pipelines 1 and 2 — Amazon seeds with NumberSells — the system produces estimates in *absolute units*. When absolute units are available across all subcategories within a parent, aggregation is straightforward: the parent-level share of a product is simply its absolute units divided by the sum of all units across all subcategories. No additional weighting mechanism is needed.

The challenge arises specifically for **non-Amazon seeds** (Pipelines 3 and 4), which produce *relative* market share estimates: a percentage that sums to one within each subcategory PLP independently. Two subcategories with relative shares summing to 100% each cannot be combined without knowing their relative sizes with respect to each other. A subcategory with 1,000 products

at 0.1% share each is not equivalent to a subcategory with 10 products at 10% share each, even though both sum to 100%. Summing or averaging relative shares across subcategories without a weighting mechanism would produce a meaningless number.

The correct aggregation requires a measure of the *revenue weight* of each subcategory within the parent category. Since actual revenue figures are not publicly available for non-Amazon retailers, the *revenue proxy* is introduced to serve this role.

6.4.2 The Revenue Proxy: Definition and Empirical Basis

The revenue proxy is defined at the product level as the number of new customer reviews added during the last week (the weekly review delta). Its rationale rests on an empirical finding: the share of weekly reviews accumulated by a subcategory within its parent approximates the share of weekly revenue generated by that subcategory within the parent. This was verified in the study described in Section 6.4.5.

Two properties of the revenue proxy are critical to its correct use:

1. It is designed to be used **always at aggregated level** — summed across all products within a subcategory or category. It is never meaningful at the individual product (SKU) level.

2. It serves two distinct purposes: **enabling revenue comparison across weeks** (the weekly delta is comparable across time, unlike cumulative review counts) and **enabling market share aggregations across the category hierarchy**.

6.4.3 Aggregation Formula

Let $\mathcal{C}$ be a parent category containing subcategories $s_1, s_2, \ldots, s_k$. For each product i in subcategory s, let $MS_i^{(s)}$ denote its market share within s (so $\sum_{i \in s} MS_i^{(s)} = 1$) and R_i its revenue proxy (weekly review delta). Define the total revenue proxy for subcategory s as $R_s = \sum_{i \in s} R_i$, and the total for the parent as $R_{\mathcal{C}} = \sum_s R_s$.

The market share of product i (from subcategory s) at the parent category level is:

$$MS_i^{(\mathcal{C})} = MS_i^{(s)} \times \frac{R_s}{R_{\mathcal{C}}} \tag{6.5}$$

That is, each product's subcategory market share is rescaled by the fraction of total parent-category revenue proxy contributed by its subcategory. Products from a subcategory that accounts for 80% of the category's review delta will have their market shares scaled by 0.80; products from a subcategory accounting for 20% will be scaled by 0.20. The resulting category-level market shares automatically sum to 1 across all products in the parent category: $\sum_s \sum_{i \in s} MS_i^{(\mathcal{C})} = \sum_s \frac{R_s}{R_{\mathcal{C}}} = 1$.

6.4.4 Worked Example

Table 6.5 illustrates the aggregation with a concrete example. Two subcategories — *Soft Drinks* (three products) and *Water* (two products) — belong to the parent category *Beverages*. Market share within each subcategory is given by the model; market share at the Beverages level is computed using the revenue proxy.

Table 6.5: Market share aggregation from subcategory to category level using the revenue proxy. The total revenue proxy for Soft Drinks is $R_{\text{SD}} = 400 + 300 + 100 = 800$ and for Water is $R_{\text{W}} = 150 + 50 = 200$, giving a category total of $R_{\mathcal{C}} = 1{,}000$. Category-level market share is computed with Eq. (6.5).

ASIN	Subcategory	Revenue proxy	Subcat. MS (model output)	Scaling factor $R_s/R_{\mathcal{C}}$	Category MS (computed)
B00E22UKVG	Soft Drinks	400	0.70	$800/1000 = 0.80$	0.56
B000R9AK5O	Soft Drinks	300	0.25	$800/1000 = 0.80$	0.20
B000R9AKYU	Soft Drinks	100	0.05	$800/1000 = 0.80$	0.04
B01E6WOVX4	Water	150	0.60	$200/1000 = 0.20$	0.12
B089FLWNP3	Water	50	0.40	$200/1000 = 0.20$	0.08
Check: category-level shares sum to					**1.00**

The key observation is that Soft Drinks accounts for 80% of the Beverages revenue proxy (800/1000), so all Soft Drinks market shares are scaled by 0.80, and Water accounts for the remaining 20%, so Water market shares are scaled by 0.20. The scaling does not change the *relative* ordering of products within a subcategory — a product at 70% subcategory share still dominates a product at 25% — but it converts all products onto a common category-level scale. The category-level shares are computed at the dashboard or ETL layer and are not produced by the pipeline itself; the pipeline emits the subcategory market shares and the per-product revenue proxy values that make the downstream computation possible.

6.4.5 Empirical Validation of the Revenue Proxy

The aggregation methodology rests on the assumption that review-share approximates revenue-share at the subcategory level. This was tested in an offline correlation study using four weeks of production data (2026-02-03 to 2026-02-24), covering 2,810,787 product rows across multiple stores and category hierarchies.

For each subcategory-parent pair identified in the data, two ratios were computed: the ratio of predicted market share in the subcategory to the parent total (market share ratio), and the ratio of the subcategory's revenue proxy sum to the parent total (revenue proxy ratio). Two analyses were conducted.

Analysis 1: using observed parent category listings. In this analysis the parent category's totals are computed from the products that actually appear in the parent PLP. Over 118 subcategory-parent seed pairs across four dates, the Pearson correlation between market share ratio and revenue proxy ratio was 0.741 after outlier removal. The weighted correlation (weighting pairs by seed size) ranged from 0.546 to 0.950 across dates, with a global weighted value of 0.675.

Analysis 2: parent redefined as the sum of its subcategories. Because not all subcategory products appear in the parent PLP (Section 3.3.2), a second analysis redefines the parent's totals as the sum of all its children. This mirrors the intended aggregation in production and requires at least two child subcategories per parent, yielding 86 pairs. The results are reported in Table 6.6.

Under Analysis 2, which matches the production aggregation model, the global correlation rises to 0.818 and exceeds 0.90 in three of the four weeks. The lower value on 2026-02-17 (0.594) reflects

Table 6.6: Pearson correlation between market share ratio and revenue proxy ratio by date (parent redefined as sum of subcategories, Pipelines 1 and 2 seed pairs). Global correlation across all 86 pairs: **0.818**.

Date	N pairs	Correlation
2026-02-03	27	0.904
2026-02-10	16	0.954
2026-02-17	27	0.594
2026-02-24	16	0.977
Global (all dates)	86	**0.818**

an outlier week; the remaining three weeks show strong alignment between review-share and revenue-share ratios. These results empirically validate the aggregation methodology: summing weekly review deltas by subcategory provides a reliable approximation of the revenue weighting needed to aggregate market shares upward in the category hierarchy.

6.4.6 Implementation

The revenue proxy is computed per product in Step 5 of the Pipeline. A *decreasing isotonic regression* is fitted mapping BSR rank to weekly review delta, trained only on products with positive review deltas and non-zero rank, with sample weights 1+100/rank to emphasise top-ranked products. A tail anchor at the maximum rank with target $\varepsilon = 0.001$ enforces monotone decay to zero. If no positive review deltas exist in the execution, all products receive `REVENUE_PROXY` $= 1.0$ (equal weight).

The resulting per-product revenue proxy value is emitted in the output contract alongside the market share prediction. Downstream, the values are aggregated at subcategory level to compute R_s, and the category-level market share is derived using Eq. (6.5). This computation takes place in the dashboard or ETL layer rather than inside the pipeline itself.

Chapter 7

Monitoring, Testing and Evaluation

This chapter covers the operational observability of the market share pipeline: how execution is monitored, how the code is tested, and how the architectural redesign described in Chapter 5 is quantitatively evaluated in terms of execution performance and cost.

7.1 Monitoring Design

Monitoring the pipeline requires observability at three distinct layers: the infrastructure level (is the execution completing successfully?), the data quality level (is the input data healthy?), and the model output level (are the predictions behaving as expected?). The monitoring strategy described here covers the estimation execution pipeline, which falls within the scope of this work. A prerequisite that is assumed but not part of this scope is the correct operation of the upstream scraping layer: for the estimation results to be meaningful, the PLP scraping must have completed successfully for all active seeds in the relevant date window. Monitoring the scraping process is maintained separately and is a dependency of the estimation pipeline rather than a component of it.

7.1.1 Execution Logs

Every invocation of Pipeline Complete writes a structured execution log to cloud storage. The log captures, for each seed-and-location combination: the tier assigned by the orchestrator, the pipeline routed to, wall-clock time per step, and peak resident set size (RSS) per execution phase. These logs are produced by the timing instrumentation (`step_timing.py`) and the execution log module, which are active in both development and production runs.

The logs serve two immediate purposes. First, they provide a complete audit trail: any prediction in the output can be traced back to the specific Airflow run, Fargate task, seed, and pipeline that produced it, via the `RUN_ID` and `AI_EXECUTION_ID` fields propagated through every output row. Second, they expose the memory footprint of each batch, enabling the team to identify clients or batches that approach the Fargate task memory limit and adjust the batch-index configuration accordingly.

7.1.2 CloudWatch Infrastructure Monitoring

At the infrastructure level, AWS CloudWatch monitors the Fargate task executions triggered by the Airflow child DAG. Metrics tracked include task success and failure rates, task duration per client-and-batch combination, and container-level CPU and memory utilisation. CloudWatch alarms are configured to notify the team when tasks fail or when execution times deviate significantly from the expected weekly window, triggering the alerting step (Task 6) in the parent DAG (Section 5.3.1).

7.1.3 Model Output Monitoring

At the model output level, monitoring focuses on detecting distribution shifts in the predictions: unusual changes in the distribution of estimated market shares, unexpected tier assignments, or anomalous revenue proxy values. This layer is currently in a design phase. The designed approach involves computing distribution-level statistics (including the Population Stability Index, PSI) over rolling weekly windows and flagging executions where the output distribution diverges significantly from the historical baseline. These checks would be run as part of the downstream DBT refresh step and surface alerts in the Airflow notification task.

7.2 Testing

The market share estimation codebase employs a step-oriented unit testing strategy implemented with Python's standard `unittest` framework. Tests live directly in the `marketshare` repository and are executed automatically in GitHub Actions on every pull request targeting the `main` or `develop` branches. A pull request cannot be merged if any test fails.

Rather than testing the full end-to-end workflow, the suite isolates individual pipeline stages. Each step function is fed a DataFrame that mimics the output of its immediate predecessor — either a frozen CSV snapshot from a real production execution or a programmatically constructed minimal frame for cases that require fine-grained control over signal availability. A shared bootstrap module stubs the cloud storage layer and loads the production configuration file, so tests run against the same tier-to-pipeline routing rules as the live system without requiring access to Snowflake, cloud storage, or any trained model artifacts.

Assertions focus on **schema contracts and deterministic business rules** rather than on model accuracy. The import step is tested to verify that the output conforms to the expected column schema and that the step is a non-destructive passthrough of the bulk-loaded data. The orchestrator step is the most thoroughly tested: it exercises all nine execution tiers, covering every combination of retailer type, rank availability, NumberSells presence, Vendor Central actuals, and review and rating signal coverage. For each tier, the test checks that the correct tier index and pipeline identifier are assigned, and verifies that tiers without an assigned pipeline raise an appropriate error. The inference step is tested for schema conformance and for correct flag assignment on the absolute-sales path. The results step tests that inference columns are correctly reshaped into market share outputs, that Amazon actuals overrides are applied in the right priority order (manufacturing view over sourcing view, and both over model predictions), and that the final output contract has the structure required for downstream ingestion.

The feature engineering, prediction, and evaluation steps are not currently covered by unit tests. This reflects a deliberate, interface-first philosophy: the correctness of numerical model outputs is addressed through offline evaluation on held-out production data (Sections 6.3.3 and 6.3.4), while the unit test suite concentrates on structural integrity and routing correctness — the failure modes most likely to be introduced by refactoring or schema changes.

7.3 Execution Performance Evaluation

The primary quantitative evaluation of the architectural redesign is execution performance and cost. These metrics directly determine whether the system is viable for weekly production delivery at RMM scale.

7.3.1 Observed Cost Reduction Across Architectural Generations

Performance and cost were measured from real production Friday runs for the RMM client across the three architectural generations. Table 7.1 summarises the results.

Table 7.1: Observed weekly production performance and cost for the RMM client across the three architectural generations. Times are total DAG execution times. Costs cover ECS/Fargate and Snowflake; S3 storage is negligible. System 0 did not complete (AWS 12-hour timeout); its cost generated no useful output.

Architecture	DAG time	Snowflake cost	ECS cost
System 0 (original)	>12 h (timeout)	$95	$283
System 1 (intermediate)	2–3 h	$41	$155
System 2 (current)	≈1 h	$32	$82
Reduction (S0 → S2)	**>12×**	**66%**	**71%**

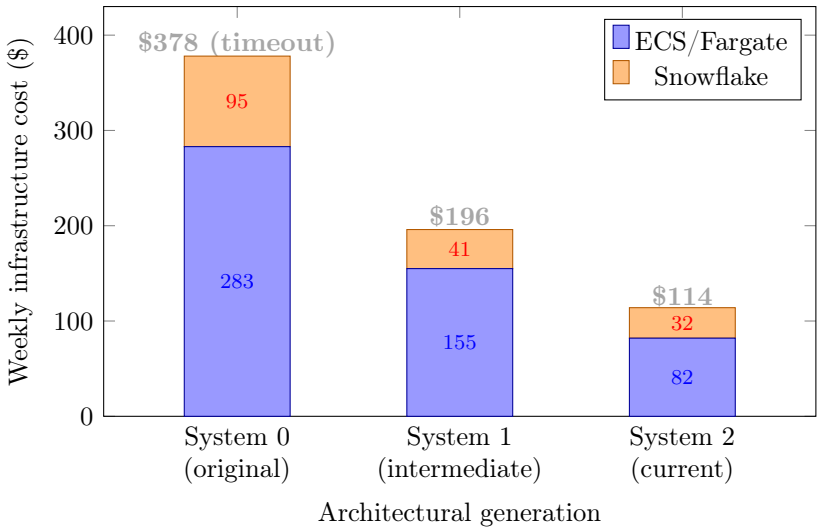

Figure 7.1: Weekly infrastructure cost breakdown by architectural generation. Each bar shows the Snowflake warehouse credit cost (orange) and the ECS/Fargate compute cost (blue). System 0 did not complete its run; the $378 figure represents cost with no useful output. The total fell from $378 to $114 — a 70% reduction — driven by the collapse in query volume and task count.

The total weekly infrastructure cost fell from $378 (System 0, aborted) to $196 (System 1) to

$114 (System 2) — a **70% reduction** over the project. Crucially, the System 0 figure of $378 was spent without producing any output, as the run aborted at the 12-hour AWS connection limit.

What changed in each generation. System 0 processed all seeds in a single Fargate task per client, issuing between 10 and 22 Snowflake queries per seed (≈13 in typical conditions). With more than 20,000 seeds, over 500,000 queries entered the shared warehouse queue — which could only process two to three simultaneously — and the execution timed out without completing.

System 1 introduced batch-level parallelism (one Fargate task per 200-seed batch, ∼60 tasks per client) and reduced per-seed queries to 1–3 by embedding dimension attributes in DBT. This brought total execution to 2–3 hours, but the Snowflake queuing problem persisted because many tasks were still issuing concurrent queries. Costs fell to $196/week.

System 2 issues one bulk query per Fargate task, covering the full weekly panel for the assigned batch of approximately 2,500 seeds. Total queries per weekly run collapsed from more than 500,000 to approximately 24. The ECS computation for a single RMM batch takes roughly 15–20 minutes; because batches run in parallel, the full weekly DAG — including the two DBT steps and all clients — completes in approximately one hour. Costs fell to $114/week.

Scalability. The cost benefit compounds as the seed universe grows. In System 2, Snowflake query count scales with the number of client-and-batch combinations, not with seed count within a batch. The current architecture is therefore cost-sublinear in seed count; System 1 was cost-linear. Adding more seeds to an existing batch costs marginal bulk-query time but no additional query slots.

7.3.2 Batch-Level Execution Detail

The production log for a single System 2 batch (2,366 seed-and-location combinations) provides a precise breakdown of time within *Pipeline Complete*, showing clearly where the remaining wall-clock time is spent after the query bottleneck was eliminated.

Table 7.2: System 2 *Pipeline Complete* phase timing for one batch of 2,366 seed-and-location combinations. Peak RSS reached 14.2 GB during the bulk import; computation phases use no additional memory after the bulk dataframe is released at the end of Phase 2.

Phase	Description	Time (s)	Peak RSS
1	Validation & setup	0.05	243.7 MB
2	Bulk Snowflake import	217.28	14,172.8 MB
3	Build slices	27.68	14,172.8 MB
4	Parallel Pipeline (2,366)	1,016.31	14,172.8 MB
5	Completion & logging	<0.01	14,172.8 MB
Total		**1,261.32 s**	**14,172.8 MB (22.3% of 63.6 GB)**

Phase 2 (bulk import, 217 s, 17% of total time) is where all data retrieval occurs and where peak memory is reached: the 13.9 GB RSS increase reflects loading the complete weekly panel for 2,366 seeds in a single `pd.read_sql` call. After Phase 2, the bulk dataframe is deleted and the garbage collector is invoked; Phases 3–5 run at constant RSS. Phase 4 (1,016 s wall-clock,

10 parallel threads, 83% of total time) contains all the estimation work. Within Phase 4, the cumulative step times across all 2,366 threads are: Step 3 feature engineering (3,859 s), Step 5 inference (2,718 s), Step 7 results (1,658 s), Step 4 prediction (680 s), Step 2 orchestration (79 s), Step 1 import sort (38 s), Step 6 evaluation (0.1 s). These are thread-summed totals; real wall-clock Phase 4 time (1,016 s) reflects the 10-thread parallel speedup.

The contrast with System 1 illustrates the architectural shift directly: in System 1, a representative batch of 143 combinations spent **5,763 s out of 5,995 s total** (96% of execution time) in Step 1 waiting for Snowflake. In System 2, the equivalent database phase accounts for 217 s out of 1,261 s (17%). System 1 spent nearly all its time waiting for the database; System 2 spends nearly all its time computing.

Chapter 8

Ethical, Legal and Sustainability Considerations

8.1 Legal Considerations in Data Collection

The primary data source of the estimation pipeline is web scraping: the automated extraction of publicly visible information from retailer websites. While scraping publicly accessible data has been upheld in landmark cases — including *hiQ Labs v. LinkedIn Corp.* (2019), which established that scraping publicly available data does not violate the Computer Fraud and Abuse Act — the legal landscape around scraping remains context-dependent and evolving.

Shalion's scraping infrastructure operates exclusively on publicly accessible pages that require no login or circumvention of technical protections. Only non-personal, product-centric information is collected: product titles, prices, rankings, review counts, and promotional flags. No user-identifiable data is processed at any stage of the estimation pipeline, placing the system outside the scope of personal data protection obligations under the GDPR [18]. The scraping process respects rate limits and does not interfere with the target infrastructure.

The Vendor Central actuals integrated in this project (Section 3.1.2) are obtained only through explicit, client-authorised API access. Access is granted per client and per account; the pipeline does not attempt to retrieve actuals for any account without the corresponding client consent. This design ensures that confidential first-party sales data is handled exclusively within the terms of the commercial agreement between the client and Amazon.

8.2 Regulatory Framework and AI Compliance

The system is classified as a minimal-risk AI application under the EU AI Act [19] (Regulation (EU) 2024/1689). The criteria for minimal risk are met on all relevant dimensions: the system does not perform biometric identification, does not operate in critical infrastructure domains, does not affect education, employment, or access to essential services, and does not make automated decisions with legal or significant personal effects. Its outputs — estimated market shares and sales volumes at product and brand level — are analytical indicators used by brands for competitive benchmarking and planning, not binding determinations.

The system's outputs are framed explicitly as estimates derived from indirect signals, not

as primary sales data. The output contract includes metadata that identifies the modelling approach and signal conditions used to produce each prediction, enabling downstream consumers to understand the provenance and reliability of every estimate. This transparency design is consistent with the epistemic integrity principles of the OECD AI Principles and the EU's Ethics Guidelines for Trustworthy AI.

8.3 Ethical Considerations

Transparency and epistemic integrity. All outputs produced by the pipeline are clearly labelled as estimates. The tier and pipeline identifiers stored in the output contract allow any stakeholder to understand which modelling approach produced a given prediction and under what signal conditions. This audit trail is a deliberate design choice, enabling informed use of the estimates and discouraging the conflation of model output with observed fact.

Fairness across brands and retailers. The estimation logic relies exclusively on public-facing, objective signals (rank, units-sold indicators, reviews, ratings). It does not incorporate any proprietary data about specific brands or any signal that could systematically advantage or disadvantage a particular company. The tier system routes all seeds through the same decision function regardless of the brand or retailer involved.

Purpose limitation. The system is designed for competitive market analysis — understanding the relative performance of products and brands within a category. It is not designed, and should not be used, to support anti-competitive behaviour such as coordinated pricing, predatory strategies, or market manipulation. The estimates describe historical patterns; they are not intended as inputs to automated pricing or bidding systems that could affect market dynamics.

8.4 Sustainability Considerations

Algorithmic efficiency (Green AI). The modelling choices in this thesis deliberately favour lightweight, computationally efficient algorithms: isotonic regression and gradient-boosted trees (XGBoost). These methods achieve the estimation accuracy required by the use case without the energy cost associated with deep learning architectures that would require GPU training and large-scale inference [8], [15]. However, the most significant sustainability contribution of this work is not the choice of algorithm but the architectural redesign of the data retrieval layer: reducing weekly Snowflake query volume from more than 500,000 individual reads to approximately 24 bulk queries per weekly run across all clients (of which roughly 10 belong to the RMM client) eliminates the dominant source of wasteful compute in the original system.

Reduction in query volume and execution time. The primary sustainability impact comes from eliminating the query bottleneck that caused System 0 to exceed the 12-hour AWS connection limit without producing output. Table 8.1 quantifies the reduction in execution time and query count across the three architectural generations, using the observed production figures from Section 7.3.

Serverless execution model. The pipeline runs once per week on AWS Fargate, a serverless platform that allocates compute resources only during the execution window and releases them immediately on completion. There is no idle server consumption between weekly runs, which

Table 8.1: Execution time and query volume reduction across architectural generations for the RMM client. System 0 produced no output (timeout). All figures are observed production values or derived from them.

Architecture	DAG time	Snowflake queries/week	ECS cost/week
System 0 (original)	>12 h (aborted)	>500,000	$283
System 1 (intermediate)	2–3 h	∼60,000	$155
System 2 (current)	≈1 h	∼24	$82
Reduction S0 → S2	**>12×**	**>20,000×**	**71%**

eliminates a category of energy waste that would exist in a permanently running virtual machine model.

Quantitative energy and carbon reduction. Rather than using cost as a proxy, the energy calculation is based directly on the number of ECS/Fargate tasks launched and their observed wall-clock duration for the RMM client — the dominant workload. Other clients are significantly smaller and do not materially affect the aggregate. Each Fargate task is configured with 4 vCPUs. The derivation uses: (1) 10 W per vCPU following the methodology of Lannelongue et al. [20]; (2) a data centre PUE of 1.2, consistent with Amazon's published sustainability figures [21]; and (3) the US average grid carbon intensity of 0.386 kg CO_2e/kWh [22].

Step 1: total vCPU-hours per weekly run.

$$\begin{aligned}
&\text{System 0:} && 1\,\text{task} \times 12\,\text{h} \times 4\,\text{vCPU} = \mathbf{48}\,\text{vCPU-h} \quad \text{(no output produced)}\\
&\text{System 1:} && 60\,\text{tasks} \times 2.5\,\text{h} \times 4\,\text{vCPU} = \mathbf{600}\,\text{vCPU-h}\\
&\text{System 2:} && 10\,\text{tasks} \times 0.5\,\text{h} \times 4\,\text{vCPU} = \mathbf{20}\,\text{vCPU-h}
\end{aligned}$$

Step 2: energy consumed ($E = \text{vCPU-h} \times 0.010\,\text{kW} \times \text{PUE}$).

$$\begin{aligned}
&\text{System 0:} && E_0 = 48 \times 0.010 \times 1.2 = \mathbf{0.58}\,\text{kWh}\\
&\text{System 1:} && E_1 = 600 \times 0.010 \times 1.2 = \mathbf{7.20}\,\text{kWh}\\
&\text{System 2:} && E_2 = 20 \times 0.010 \times 1.2 = \mathbf{0.24}\,\text{kWh}
\end{aligned}$$

Step 3: carbon emissions ($C = E \times 0.386\,\text{kg CO}_2\text{e/kWh}$).

$$\begin{aligned}
&\text{System 0:} && C_0 = 0.58 \times 0.386 \approx \mathbf{0.22}\,\text{kg CO}_2\text{e} \quad \text{(no output)}\\
&\text{System 1:} && C_1 = 7.20 \times 0.386 \approx \mathbf{2.78}\,\text{kg CO}_2\text{e}\\
&\text{System 2:} && C_2 = 0.24 \times 0.386 \approx \mathbf{0.09}\,\text{kg CO}_2\text{e}
\end{aligned}$$

System 2 emits approximately **0.09 kg CO_2e per weekly run**. System 1 was the most energy-intensive generation because 60 parallel tasks each ran for 2.5 hours, totalling 600 vCPU-hours and **2.78 kg CO_2e/week**. System 2 processes the same seed universe in 10 tasks of 30 minutes each — a **97% reduction in both energy and carbon** relative to System 1. System 0's 48 vCPU-hours were consumed without producing any output, making its effective carbon cost per delivered prediction undefined. Figure 8.1 summarises these results visually.

Alignment with the UN Sustainable Development Goals. The project maps to SDG 9

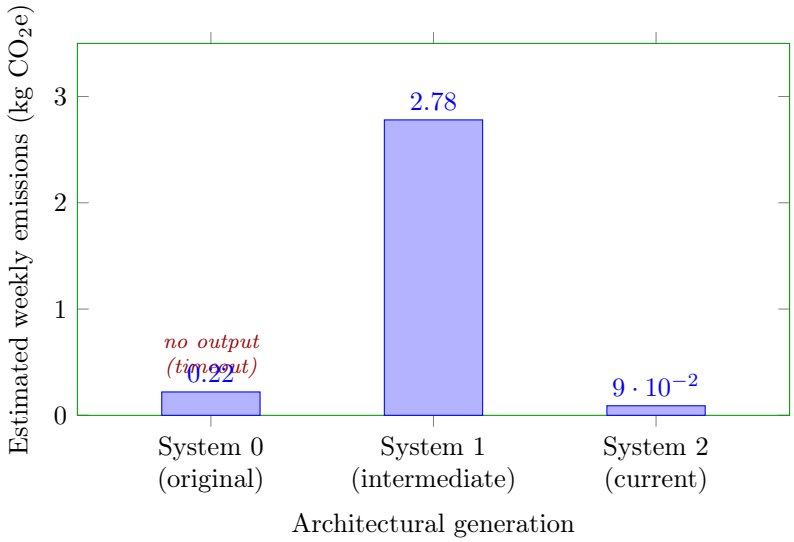

Figure 8.1: Estimated weekly carbon footprint by architectural generation (kg CO_2e). Based on observed ECS task count and wall-clock duration: System 0 (1 task, 12 h), System 1 (60 tasks, 2.5 h each), System 2 (10 tasks, 30 min each), all with 4 vCPUs per task. Assumptions: 10 W/vCPU [20], PUE 1.2 [21], US grid intensity 0.386 kg CO_2e/kWh [22]. System 1 dominates because of the large number of long-running parallel tasks.

(Industry, Innovation and Infrastructure) through the development of resilient and efficient data infrastructure; SDG 12 (Responsible Consumption and Production) through the elimination of wasted compute from aborted long-running tasks, a 70% reduction in weekly infrastructure cost, and a 97% reduction in energy and carbon relative to the most energy-intensive generation (System 1); and SDG 17 (Partnerships for the Goals) through the cross-team collaboration between AI and Data Engineering that the architectural redesign required.

Chapter 9

Discussion and Conclusions

9.1 Summary of Contributions

This thesis carried an AI-driven market share estimation service from a functional prototype into weekly production at a scale that was previously infeasible. The contributions span data engineering, system architecture, and modelling, and they are grounded in two concrete business requirements: delivering market share for RMM at more than 20,000 seeds across four US retailers, and improving estimation precision for specific client categories.

Architectural redesign. The primary contribution is the elimination of the per-seed database query bottleneck that made the baseline system untenable at scale. Through two successive refactoring stages, the total database query volume per weekly run was reduced from more than 500,000 to a handful of bulk queries per client batch, cutting the ECS computation time for a single RMM batch (approximately 2,500 seeds) from being unable to complete within the 12-hour limit to 15–20 minutes. Because batches run in parallel, the full weekly DAG now completes in approximately one hour, down from a System 0 run that never finished. The redesign introduces a clean separation between *Pipeline Complete* (responsible for all data access) and *Pipeline* (responsible for pure in-memory computation), a pattern that scales horizontally via the Airflow child DAG's dynamic task mapping.

Scraping strategy and data alignment. The data collection cadence was redesigned to align with natural calendar weeks, ensuring that estimates correspond to a clean Monday-to-Sunday period. The date-tolerance mechanism, previously implemented as a Python retry loop issuing up to twelve sequential queries, was moved into the SQL bulk query as a single range filter with in-memory date selection, eliminating both the query overhead and the code complexity.

Extended tier system and new modeling pipelines. The original four-scenario orchestration was extended to a nine-tier system covering non-Amazon retailers, rank-only seeds, and engagement-poor environments. Two new XGBoost-based pipelines were trained, evaluated, and deployed: Pipeline 3 (rank + engagement features) achieves a WMAPE of 45.9% on held-out data, compared to 101.4% for the legacy model, a 55% improvement; Pipeline 4 (rank only) achieves a WMAPE of 63.0% in rank-only conditions, compared to 115.0% for the legacy approach. The routing boundary between the two models was derived from a systematic null-rate injection experiment rather than by manual threshold selection.

Revenue proxy and aggregation methodology. A mechanism was developed and

empirically validated to aggregate subcategory-level market shares upward in the category hierarchy using weekly review deltas as a revenue weight proxy. The Pearson correlation between review-share and revenue-share ratios was 0.818 on held-out data under the production aggregation model, validating the approach as a practical solution to a problem that could not be solved by naive summation.

Development environment and designed Iceberg migration. A two-environment development and production infrastructure was established, enabling safe iteration on the live system. The migration to Apache Iceberg table format was designed, implemented on a development branch, and validated end-to-end, with the production blocker (concurrent-write conflicts under parallel Fargate execution) identified and three candidate resolutions scoped.

9.2 Practical Implications

The system described in this thesis runs in production and serves real clients weekly. This is not a prototype or a proof of concept: the market share estimates it generates are consumed by Shalion's clients through the RMM and MSM products, and the roughly one-hour weekly DAG runtime — with each RMM batch computing in 15–20 minutes in parallel — means the full weekly run fits comfortably within the Friday production window even as the seed universe continues to grow.

The practical implications extend beyond the immediate use case. The architectural pattern — bulk SQL to retrieve the full estimation panel, in-memory parallel computation, S3 output contracts consumed by downstream DBT models — is a general-purpose design for batch ML inference at scale that is applicable to any system where per-record database access is the dominant cost. The tier-based routing pattern is equally general: any multi-signal estimation problem where signal availability varies by observation benefits from routing each observation to the model best suited to its specific signal environment, rather than applying a single universal model that degrades gracefully but uniformly.

9.3 Societal and Business Impact

Business impact. The most direct business impact of this thesis is enabling a new commercial product. The RMM market share module could not serve clients before this work; it can now. This creates a direct revenue opportunity for Shalion through the market share component of the RMM subscription, and it strengthens the competitive positioning of the product in a market where data latency and granularity are key differentiators. The improvement in Pipeline 3 and 4 accuracy (WMAPE from 101% to 46% and from 115% to 63%, respectively) directly improves the quality of market intelligence delivered to clients, enabling more confident commercial decisions.

Sectorial impact. At a broader level, the system contributes to a more transparent and accessible view of product performance in e-commerce markets. Brands that previously had to rely on expensive panel data or opaque retailer reports can now access granular, weekly, product-level market share estimates derived entirely from public signals. This democratization

of competitive intelligence is particularly valuable for mid-sized brands that cannot afford the subscription cost of traditional panel data vendors.

9.4 Limitations

Ground-truth scarcity and evaluation blind spots. Vendor Central actuals are available only for a small subset of Amazon seeds belonging to clients who have granted access. For the vast majority of seeds — and for all RMM seeds by construction — there is no ground truth against which to validate predictions. Evaluation is therefore restricted to the data-rich minority, introducing survivorship bias: the measured accuracy is likely better than the true accuracy in the full production universe. Pipelines 3 and 4 are evaluated on Pipeline 1 pseudo-labels, meaning that evaluation errors are correlated with training errors in ways that a truly independent benchmark would not exhibit.

Pseudo-label dependency and error propagation. Pipelines 3 and 4 are trained on outputs from Pipeline 1 as pseudo-labels. Any systematic bias in Pipeline 1 predictions is inherited by the downstream models, which learn to mimic the teacher's behaviour — including its errors. A concrete risk is category skew: if Pipeline 1 trains primarily on one type of product category (beverages, personal care) because that is where the VC-integrated clients operate, the pseudo-labels may not represent the full diversity of rank distributions encountered in the RMM universe.

Retailer platform drift. The scraped signals are controlled by the retailers, not by Shalion. Amazon, Walmart, Target, and Kroger can change the way they display ranking information, modify the frequency of BSR updates, alter the format of the NS indicator, or discontinue certain signals entirely without notice. Any such change can silently degrade model inputs in ways that are not immediately visible in prediction outputs, making continuous monitoring of input distributions — not just output distributions — a critical but currently underdeveloped capability.

Ranking algorithm bias. The BSR is computed by each retailer's own ranking algorithm, which can incorporate factors beyond current-week sales: promotional boosts, freshness bonuses for new products, or penalisations for out-of-stock states. These algorithmic factors introduce systematic distortions between the observable rank and the underlying sales volume that the model cannot disentangle. They are particularly problematic for promoted products (which appear higher than their sales would justify) and for recently launched products (which may receive algorithmic boosts during their launch window).

Review signal volatility. The revenue proxy relies on weekly review deltas, which are sparse (many products receive zero reviews in any given week) and can be influenced by factors unrelated to sales (promotional review campaigns, negative review sprees, or retailer-side review filtering). The validation study showed one week (2026-02-17) with correlation 0.594, substantially below the other three weeks, suggesting non-negligible sensitivity to weekly anomalies that the current implementation does not smooth out. A further caveat is that the proxy was validated on Pipelines 1 and 2 (Amazon) seed pairs, yet it is applied to aggregate non-Amazon (Pipelines 3 and 4) shares, where review behaviour may differ; confirming the correlation directly on non-Amazon retailers is left as future validation work.

Coverage gap at Tiers 3 and 4. Amazon seeds with rank but without NumberSells

currently produce no output. At RMM scale, this covers a non-trivial fraction of the seed universe and represents a systematic blind spot for products that do not display a units-sold indicator.

Generalization to new retailers. The pipeline has been deployed and validated on Amazon, Walmart, Target, and Kroger in the US. Generalization to other retailers or geographies is non-trivial: different category taxonomies, different signal landscapes, and different BSR computation algorithms would all require recalibration of the tier boundary scores, the feature engineering steps, and potentially the model architectures. The tier system is designed to be extensible, but each new retailer requires an empirical characterization of its signal profile before the system can operate reliably.

Iceberg migration incomplete. The designed output format migration remains unmerged in production due to the concurrent-write blocker, leaving the current system dependent on External Tables with their associated manual refresh overhead and versioning limitations.

9.5 Future Work

Resolving the Iceberg concurrent-write blocker. The most immediate engineering priority is merging the Iceberg branch into production. The three candidate resolutions identified in Section 5.5.4 — per-client Iceberg tables, a serialised commit queue, or deferred batch registration — should be prototyped and benchmarked in the development environment before production deployment.

Closing the Tier 3/4 coverage gap. An improved Pipeline 1 that calibrates directly against manufacturing actuals (1P + 3P) without requiring NumberSells would extend coverage to the currently unserved Amazon seeds. The manufacturing view data integrated during this project (Section 3.1.2) provides the necessary ground truth for this extension.

Retraining Pipelines 3 and 4 on improved pseudo-labels. Once Tier 3/4 coverage is established and Pipeline 1 is improved, retraining the XGBoost models on the expanded and higher-quality pseudo-label set is expected to improve their accuracy, particularly for non-Amazon seeds where the current training data is sparse.

Advanced monitoring and systematic evaluation. The monitoring strategy described in Chapter 7 is partially in design phase. Implementing PSI-based drift detection, automated weekly evaluation against any available actuals, and client-level anomaly alerting would substantially improve the system's operational maturity.

Market share forecasting. The current system estimates market share for the most recently closed week. A natural extension would be to predict market share for the following week, leveraging the weekly history panel already available in the bulk query. Time-series models or sequence-aware extensions of the existing XGBoost pipelines are the most promising starting points given the data volume and latency constraints.

9.6 Personal and Professional Reflection

This internship was an opportunity to witness, from the inside, what it takes to scale a system and bring it to production. The experience differed substantially from academic project work,

and the difference was not merely one of scale or complexity — it was one of what counts as progress.

In an academic setting, progress is measured by model accuracy, by a tighter bound, by a cleaner experiment. Here, progress meant a weekly run that completed in time for Friday delivery. It meant a Snowflake query that retrieved a client's full weekly panel in one round-trip instead of fifty thousand. It meant a Fargate task that stayed within its memory budget. The metrics that mattered were wall-clock time, query count, and RSS peak — not WMAPE, not R^2. Developing an intuition for these metrics, and learning to read a system's behaviour from its logs and CloudWatch graphs, was a new kind of competence that no coursework had required.

The project also demanded learning new tools and languages in a real-stakes context rather than a tutorial. Airflow DAGs, DBT models, PyIceberg, SQL at scale in Snowflake, Terraform-backed infrastructure, Docker and AWS ECR: each of these had to be learned well enough to contribute to production code, not just to understand it conceptually. The pace at which that learning happened was compressed by the fact that decisions had consequences — a poorly written CTE in the bulk query would slow down the entire weekly run; a misconfigured ECS task override would silently fail for one client while succeeding for others.

Working closely with the Data Engineering team was equally formative. The architecture changes described in Chapter 5 were not purely AI team decisions: the bulk query design, the Iceberg migration, the index_seed batching mechanism, and the external table refresh hooks all required back-and-forth with the Data team to align on interfaces, on schema contracts, and on the downstream implications of changes to the S3 folder structure or the Snowflake schema. Learning to scope these dependencies, to communicate them clearly across teams, and to coordinate implementation so that the production DAG was never broken during the transition, were skills that the project demanded and that no amount of solo modelling work would have developed.

The modelling work — the new pipelines, the revenue proxy, the null-rate threshold study — remained important and interesting, but it was contextualized differently than in prior academic work. A model improvement matters only if it can be shipped. And shipping a model means understanding the pipeline that will serve it, the contract it writes, the monitoring that will detect if it goes wrong, and the team that will have to maintain it after the internship ends. That larger view of what it means to build an AI system is the most lasting thing this project gave me.

9.7 Final Remarks

The central contribution of this thesis is the transformation of a research prototype into a scalable operational platform. This is not primarily a statement about model accuracy or infrastructure elegance — it is a statement about what the system can now do that it could not do before. Before this work, the market share estimation service did not finish its weekly run: it hit the AWS 12-hour connection limit and was aborted. After this work, each RMM batch computes in 15–20 minutes and the full weekly run completes in approximately one hour, serves real clients weekly, and provides a foundation on which further improvements can be built. That is the primary contribution of this thesis: the productionization of an AI system that was technically valid but operationally unscalable.

This transformation required simultaneous work at every layer of the stack. The architectural redesign — eliminating the Snowflake queuing bottleneck through a two-stage refactoring — was necessary but not sufficient. The new modelling pipelines were necessary but not sufficient. The two-environment development infrastructure, the revenue proxy, the Iceberg migration design: each piece addressed a different dimension of what it means for a system to be operationally mature. No single contribution delivered the result; the result came from their coherent combination.

The thesis also illustrates a general principle about production AI systems: the engineering work that makes a system reliable, observable, and maintainable at scale is not scaffolding around the "real" AI work — it is itself a first-class technical contribution. A system that estimates market share with 46% WMAPE and runs reliably in production is more valuable than a system that achieves 30% WMAPE but cannot complete its run. The gap between those two systems is not a modelling gap; it is an engineering gap. Closing that gap, in a real production context with real constraints and real clients, is what this thesis set out to do — and what it did.

What this project leaves behind is a platform: structured, observable, testable, running in production, maintained by a team that understands it. The coverage gap at Tiers 3 and 4 remains. The Iceberg migration is pending. Ground-truth scarcity limits evaluation. But these are well-defined problems on a solid foundation — and that, in the end, is precisely what it means to productionize an AI system.

References

[1] Statista Research Department, *E-commerce worldwide – statistics & facts*, https://www.statista.com/topics/871/online-shopping/, Accessed June 2026, 2024.

[2] L. Furriols Llimargas, "AI-Driven Estimation of Retail Media Performance Metrics Across Digital Shelf Signals," Bachelor's Thesis, Universitat Politècnica de Catalunya, Facultat d'Informàtica de Barcelona, Jan. 2026.

[3] A. T. Stephen, "The role of digital and social media marketing in consumer behavior," *Current Opinion in Psychology*, vol. 10, pp. 17–21, 2016.

[4] J. A. Chevalier and A. Goolsbee, "Measuring prices and price competition online: Amazon.com and barnesandnoble.com," *Quantitative Marketing and Economics*, vol. 1, no. 2, pp. 203–222, 2003.

[5] N. Hu, N. S. Koh, and S. K. Reddy, "Ratings lead you to the product, reviews help you clinch it? the mediating role of online review sentiments on product sales," *Decision Support Systems*, vol. 57, pp. 42–53, 2014.

[6] F. Zhu and X. Zhang, "Impact of online consumer reviews on sales: The moderating role of product and consumer characteristics," *Journal of Marketing*, vol. 74, no. 2, pp. 133–148, 2010.

[7] G. Hinton, O. Vinyals, and J. Dean, "Distilling the knowledge in a neural network," *arXiv preprint arXiv:1503.02531*, 2015.

[8] D. Sculley et al., "Hidden technical debt in machine learning systems," in *Advances in Neural Information Processing Systems (NeurIPS)*, 2015.

[9] A. Paleyes, R.-G. Urma, and N. D. Lawrence, "Challenges in deploying machine learning: A survey of case studies," *ACM Computing Surveys*, vol. 55, no. 6, pp. 1–29, 2022.

[10] A. Ng, *A chat with andrew on MLOps: From model-centric to data-centric AI*, DeepLearning.AI event, Available at https://www.deeplearning.ai/, 2021.

[11] Feast Authors, *Feast: Open source feature store for machine learning*, https://feast.dev/, Accessed 2026, 2023.

[12] M. Kleppmann, *Designing Data-Intensive Applications*. O'Reilly Media, 2017.

[13] Apache Software Foundation, *Apache Iceberg documentation*, https://iceberg.apache.org/, Accessed 2026, 2026.

[14] T. Chen and C. Guestrin, "XGBoost: A scalable tree boosting system," in *Proceedings of the 22nd ACM SIGKDD International Conference on Knowledge Discovery and Data Mining*, 2016, pp. 785–794.

[15] L. Grinsztajn, E. Oyallon, and G. Varoquaux, "Why tree-based models still outperform deep learning on tabular data," in *Advances in Neural Information Processing Systems (NeurIPS)*, 2022.

[16] T. Akiba, S. Sano, T. Yanase, T. Ohta, and M. Koyama, "Optuna: A next-generation hyperparameter optimization framework," in *Proceedings of the 25th ACM SIGKDD International Conference on Knowledge Discovery and Data Mining*, 2019.

[17] N. Duan, "Smearing estimate: A nonparametric retransformation method," *Journal of the American Statistical Association*, vol. 78, no. 383, pp. 605–610, 1983.

[18] European Parliament and Council, *Regulation (eu) 2016/679 (general data protection regulation)*, Official Journal of the European Union, 2016.

[19] European Parliament and Council, *Regulation (eu) 2024/1689 laying down harmonised rules on artificial intelligence (ai act)*, Official Journal of the European Union, 2024.

[20] L. Lannelongue, J. Grealey, and M. Inouye, "Green algorithms: Quantifying the carbon footprint of computation," *Advanced Science*, vol. 8, no. 12, p. 2 100 707, 2021. DOI: 10.1002/advs.202100707.

[21] Amazon Web Services, *AWS 2023 Sustainability Report*, https://sustainability.aboutamazon.com/, Accessed June 2026, 2023.

[22] U.S. Environmental Protection Agency, *eGRID2022: Emissions & Generation Resource Integrated Database*, https://www.epa.gov/egrid, Accessed June 2026, 2023.